VOL. 14

PHP

From the built-in server to a tested, secure application

Source	phptherightway.com
Generated	2026-10-10
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 14 - PHP

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - Modern PHP setup and the built-in server 9

1.1 Summary 9

1.2 Concepts 10

One language, several server interfaces 10

The life of a request 10

Extensions you will meet 10

The built-in web server 11

Useful command-line flags 11

1.3 Examples 11

1.4 Common mistakes and best practices 13

1.5 Practice exercises 13

Solutions 13

1.6 Mini test 14

Answer key and explanations 14

Chapter 2 - Project structure, coding standards and autoloading 14

2.1 Summary 15

2.2 Concepts 15

A layout that scales 15

PSR-1 in one table 15

PSR-12 in one table 16

PSR-4: from class name to file 16

How autoloading works 16

2.3 Examples 17

2.4 Common mistakes and best practices 18

2.5 Practice exercises 19

Solutions 19

2.6 Mini test 19

Answer key and explanations 20

Chapter 3 - Object-oriented PHP 20

3.1 Summary 20

3.2 Concepts . . . 21
The anatomy of a class . . . 21
Types in signatures . . . 21
Interfaces, abstract classes and traits . . . 22
Enums . . . 22
`match` versus `switch` . . . 22
3.3 Examples . . . 22
3.4 Common mistakes and best practices . . . 25
3.5 Practice exercises . . . 26
Solutions . . . 26
3.6 Mini test . . . 26
Answer key and explanations . . . 27
Chapter 4 - Namespaces, the SPL and CLI scripting . . . 27
4.1 Summary . . . 27
4.2 Concepts . . . 27
Namespaces in one table . . . 27
Name resolution in practice . . . 28
SPL structures worth knowing . . . 28
Generators . . . 28
The CLI environment . . . 29
Xdebug modes . . . 29
4.3 Examples . . . 29
4.4 Common mistakes and best practices . . . 32
4.5 Practice exercises . . . 32
Solutions . . . 33
4.6 Mini test . . . 33
Answer key and explanations . . . 33
Chapter 5 - Composer and dependency management . . . 33
5.1 Summary . . . 34
5.2 Concepts . . . 34
The manifest, section by section . . . 34
Version constraints . . . 34
The commands you will actually type . . . 35
Lock file, vendor, and reproducibility . . . 35
Scripts . . . 36

5.3 Examples . . . 36
5.4 Common mistakes and best practices . . . 38
5.5 Practice exercises . . . 38
Solutions . . . 38
5.6 Mini test . . . 39
Answer key and explanations . . . 39
Chapter 6 - Design patterns and dependency injection . . . 39
6.1 Summary . . . 39
6.2 Concepts . . . 40
Four patterns, in the order they appear in real code . . . 40
Repository . . . 40
Strategy . . . 40
Dependency injection, by hand . . . 41
Containers . . . 41
6.3 Examples . . . 41
6.4 Common mistakes and best practices . . . 46
6.5 Practice exercises . . . 46
Solutions . . . 46
6.6 Mini test . . . 47
Answer key and explanations . . . 47
Chapter 7 - Databases with PDO . . . 47
7.1 Summary . . . 47
7.2 Concepts . . . 48
Connecting . . . 48
Attributes worth setting . . . 48
Fetch modes . . . 48
Why prepared statements are not optional . . . 49
Transactions . . . 49
7.3 Examples . . . 49
7.4 Common mistakes and best practices . . . 52
7.5 Practice exercises . . . 53
Solutions . . . 53
7.6 Mini test . . . 53
Answer key and explanations . . . 53
Chapter 8 - Templating, escaping, UTF-8 and dates . . . 54

8.1 Summary . . . 54
8.2 Concepts . . . 54
Plain PHP templates . . . 54
The render helper . . . 55
Escaping rules . . . 55
Twig syntax . . . 55
Multibyte text . . . 56
Dates . . . 56
8.3 Examples . . . 56
8.4 Common mistakes and best practices . . . 59
8.5 Practice exercises . . . 60
Solutions . . . 60
8.6 Mini test . . . 60
Answer key and explanations . . . 61
Chapter 9 - Errors and exceptions . . . 61
9.1 Summary . . . 61
9.2 Concepts . . . 62
The hierarchy . . . 62
Handling . . . 62
Custom exceptions . . . 62
Global handlers . . . 63
Reporting configuration . . . 63
9.3 Examples . . . 63
9.4 Common mistakes and best practices . . . 66
9.5 Practice exercises . . . 66
Solutions . . . 66
9.6 Mini test . . . 67
Answer key and explanations . . . 67
Chapter 10 - Security in practice . . . 67
10.1 Summary . . . 67
10.2 Concepts . . . 68
Filtering and validation . . . 68
Password hashing . . . 68
The CSRF round trip . . . 68
Sessions . . . 69

Configuration and error reporting . . . 69
10.3 Examples . . . 69
10.4 Common mistakes and best practices . . . 72
10.5 Practice exercises . . . 72
Solutions . . . 72
10.6 Mini test . . . 73
Answer key and explanations . . . 73
Chapter 11 - Testing with PHPUnit . . . 73
11.1 Summary . . . 73
11.2 Concepts . . . 74
Kinds of tests . . . 74
Layout . . . 74
Assertions you will use first . . . 74
Doubles . . . 75
11.3 Examples . . . 75
11.4 Common mistakes and best practices . . . 78
11.5 Practice exercises . . . 79
Solutions . . . 79
11.6 Mini test . . . 79
Answer key and explanations . . . 79
Chapter 12 - Deployment, caching and the ecosystem . . . 80
12.1 Summary . . . 80
12.2 Concepts . . . 80
Deployment steps . . . 80
The caching layers . . . 81
OPcache in one table . . . 81
Virtualization and containers . . . 81
PHPDoc . . . 81
The ecosystem . . . 82
12.3 Examples . . . 82
12.4 Common mistakes and best practices . . . 84
12.5 Practice exercises . . . 84
Solutions . . . 84
12.6 Mini test . . . 85
Answer key and explanations . . . 85

Chapter 13 - Capstone: a links manager 85
13.1 Summary 85
13.2 Concepts 86
The request path 86
The file layout 86
What comes from where 87
The route table 87
13.3 Examples 88
13.4 Common mistakes and best practices 96
13.5 Practice exercises 97
Solutions 97
13.6 Mini test 98
Answer key and explanations 98
Cheat sheet 98
14.1 Command line 98
14.2 Language essentials 99
14.3 Strings, arrays and dates 99
14.4 Composer 100
14.5 PDO 100
14.6 Errors, testing, security, deployment 101
Final test 101
15.1 Final test answers 102
Sources 103

How to use this volume

Prerequisites. You should be comfortable writing small programs in any language, and you should know basic HTML. No PHP experience is assumed, but you will move faster if you have already met typed variables, functions and classes somewhere else. Vol. 03 (JavaScript) or Vol. 10 (SQL) are good companions, because PHP spends most of its life between a database and a web page.

Estimated study time. About 25 hours: one to two hours per chapter plus the time you spend on your own experiments. Chapters 1 to 5 are the core language and tooling; chapters 6 to 12 cover what turns a script into an application; chapter 13 assembles everything into a working project.

How the chapters work. Every chapter opens with a summary, explains the concepts in tables and diagrams, shows numbered examples with the output they produce, lists common mistakes with the best practice for each, and closes with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Blocks tagged `php` are complete scripts you can save as `script.php` and start with `php script.php`. Blocks tagged `bash` are terminal commands, `json`, `ini`, `xml`, `sql` and `dockerfile` are configuration and schema files, and blocks tagged `text` show what a command or a script prints. Blocks tagged `twig` are template files.

shell

```
php -v
php -S localhost:8000 -t public
```

output

```
PHP 8.3.12 (cli) (built: Oct  3 2025 10:18:33) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.0.0-dev, with Zend OPcache v8.3.12
```

The exact version, build date and extension list differ on every machine. Output blocks in this book are illustrative: they show what the code prints on a current PHP 8.3 install, not the result of a run performed while writing.

About the source. The syllabus follows the community guide published at phptherightway.com, which collects the practices most PHP projects share. The guide is a reference rather than a course, so this volume reorders its topics into a learning path, adds the language details the guide assumes, and credits every page consulted in the Sources list.

Chapter map

#	Chapter	You will learn
1	Modern PHP setup and the built-in server	versions, extensions, the CLI, `php -S`, router scripts
2	Project structure, coding standards and autoloading	directory layout, PSR-1, PSR-12, PSR-4, `spl_autoload_register`

#	Chapter	You will learn
3	Object-oriented PHP	typed properties, promotion, interfaces, traits, enums, readonly, `match`
4	Namespaces, the SPL and CLI scripting	`namespace`, `use`, stacks and queues, generators, `$argv`, Xdebug
5	Composer and dependency management	`composer.json` fields, version constraints, scripts, the lock file
6	Design patterns and dependency injection	repositories, strategies, factories, containers, why not to over-abstract
7	Databases with PDO	DSNs, prepared statements, fetch modes, transactions
8	Templating, escaping, UTF-8 and dates	plain PHP templates, `htmlspecialchars`, Twig, `mbstring`, `DateTimeImmutable`
9	Errors and exceptions	`Throwable`, custom exceptions, handlers, error-reporting setup
10	Security in practice	password hashing, filtering, CSRF tokens, sessions, configuration files
11	Testing with PHPUnit	`phpunit.xml`, assertions, data providers, test doubles
12	Deployment, caching and the ecosystem	OPcache, deploy steps, Docker, PHPDoc, PSR list
13	Capstone: a links manager	a small modern PHP application assembled from chapters 1 to 12
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Modern PHP setup and the built-in server

1.1 Summary

- PHP 8.3 is the current stable line, and each minor release adds real language features while removing the deprecated ones.
- One binary serves every request model: the command line, Apache, PHP-FPM and the built-in development server all run the same engine.
- The built-in server is a single-process tool for development; production runs PHP-FPM or an Apache module behind a real web server.
- A router script lets the development server send every non-file request to one front controller, which is how clean URLs work during development.

- The CLI flags `-l`, `-m`, `-i`, `-v` and `-S` answer the questions you ask most often while setting a project up.
- Extensions such as `pdo_sqlite`, `mbstring` and `openssl` are compiled by default on most distributions and are enabled in `php.ini` elsewhere.
- `declare(strict_types=1)` is the first statement of a file and makes scalar type declarations strict for that file only.
- Windows, macOS and Linux all ship ready-to-run packages, so the same code behaves the same everywhere.

1.2 Concepts

One language, several server interfaces

PHP normally answers web requests, but the same interpreter runs scripts. The server API (SAPI) decides how the engine is embedded:

SAPI	How it runs	Typical use
`cli`	`php script.php`	cron jobs, workers, console commands
`cli-server`	`php -S host:port`	local development
`fpm-fcgi`	PHP-FPM pool behind nginx or Apache	production
`apache2handler`	Apache module	shared hosting, older stacks

The life of a request

output

```
 browser                web server             php-fpm / cli-server
+--------+  GET /links   +--------+  index.php  +-------------------+
| sends  | -------------> | routes | ----------> | runs your script  |
| HTTP   |                | static |             | build response    |
| gets   | <------------- | files  | <---------- | echo / json       |
| HTML   |     response   | itself |             +-------------------+
+--------+                +--------+
```

Your script reads the request, touches a database or a template, writes output and ends. Nothing stays in memory between requests, so every request starts from a clean slate.

Extensions you will meet

Extension	What it adds	Usually installed
`pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`	database drivers for PDO	`pdo_sqlite` by default
`mbstring`	multibyte-safe string functions	yes
`openssl`	random bytes, hashing helpers, TLS functions	yes
`intl`	locale-aware formatting, transliteration	optional

Extension	What it adds	Usually installed
xdebug	step debugging, coverage, stack traces	development only

Check what you have with `php -m`, which prints every loaded extension.

The built-in web server

`php -S localhost:8000 -t public` serves the `public` directory. When a request matches a real file, that file is returned; otherwise PHP runs `public/index.php` inside the document root when you point the server at it. To keep full control, pass a router script instead:

shell

```shell
php -S localhost:8000 router.php
```

The router decides, for every request, whether to serve a file or to run the front controller.

Useful command-line flags

Flag	Job
`-v`	print the version and build information
`-l file.php`	parse a file and report syntax errors without running it
`-m`	list loaded extensions
`--ini`	show which configuration files are read
`-i`	dump the full configuration, like `phpinfo()`
`-d key=value`	set one `php.ini` directive for this run
`-S host:port`	start the built-in server
`-r 'code'`	run a snippet without a file

1.3 Examples

Example 1: check the version you are actually running.

shell

```shell
php -v
```

output

```
PHP 8.3.12 (cli) (built: Oct  3 2025 10:18:33) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.0.0-dev, with Zend OPcache v8.3.12
```

Two PHP binaries on one machine (one for Apache, one for the CLI) is a classic source of confusion: always check with the same entry point you develop with.

Example 2: a first script with strict types.

PHP

```php
<?php

declare(strict_types=1);

function greet(string $name): string
{
    return sprintf('Hello, %s!', $name);
}

$release = new DateTimeImmutable('2026-10-10');

echo greet('PHP 8.3'), PHP_EOL;
printf('Ready since %s%s', $release->format('Y-m-d'), PHP_EOL);
```

output

```
Hello, PHP 8.3!
Ready since 2026-10-10
```

`declare(strict_types=1)` must be the first statement in the file. With it, a call such as `greet(42)` throws a `TypeError` instead of silently converting the number to a string.

Example 3: start the development server.

shell

```
php -S localhost:8000 -t public
```

output

```
[Sat Oct 10 21:42:05 2026] PHP 8.3.12 Development Server (http://localhost:8000) started
```

Requests for existing files are served directly; everything else produces the documented "Not Found" response unless a router script takes over.

Example 4: a router script that routes unknown URLs to the front controller.

PHP

```php
<?php

declare(strict_types=1);

$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$file = __DIR__ . '/public' . $path;

if ($path !== '/' && is_file($file)) {
    return false;
}

require __DIR__ . '/public/index.php';
```

Save it as `router.php` and run `php -S localhost:8000 router.php`. Returning `false` tells the server to send the requested file unchanged, so images and style sheets keep working while every other URL reaches `index.php`.

Example 5: lint a file before running it.

shell

```
php -l src/Quote.php
```

output

```
No syntax errors detected in src/Quote.php
```

A file with a missing semicolon reports the line instead:

output

```
PHP Parse error:  syntax error, unexpected token "echo" in src/Quote.php on line 7
```

`php -l` finds parse errors only; it cannot tell you that a function call will fail at runtime.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using `php -S` for production	single process, no hardening, no TLS plan	run PHP-FPM behind nginx or Apache
Editing the wrong `php.ini`	changes seem to be ignored	run `php --ini` and edit the loaded file
Opening files with `<?` instead of `<?php`	depends on `short_open_tag`, off since PHP 5.4 habits die hard	always write `<?php`
Assuming the CLI and the web share configuration	different `php.ini` per SAPI	run `php --ini`, then compare with a `phpinfo()` page in the browser
Skipping `declare(strict_types=1)`	silent scalar coercion hides bugs	declare it in every file you write
Installing extensions globally and forgetting them	the app works locally and not on the server	list `ext-*` packages in `composer.json`

- Keep one PHP version per project, documented in `composer.json` as `php`.
- When nothing seems to happen, read the server log window, not just the page.

1.5 Practice exercises

1. Which command prints every extension loaded by your CLI PHP?
2. What does `return false` inside a router script do?
3. Why is `declare(strict_types=1)` enough for one file but not the whole project?
4. Name two command-line flags you would use to debug a script that dies before printing anything.
5. Why does the built-in server warn that it must not be used in production?

Solutions

1. `php -m`. For a single extension, `php -m | grep pdo` narrows the list.

2. It tells the built-in server to serve the requested file as-is, which keeps static assets working while other URLs reach the front controller.
3. `declare` is a file-level statement: it changes the coercion rules only for the file that contains it, so every file must declare it.
4. `php -l file.php` catches parse errors, and `php -d display_errors=1 -d error_reporting=E_ALL script.php` surfaces errors that the configuration hides. `php --ini` shows which configuration file is in charge.
5. It runs one single-threaded process with no access control, no TLS and no request limits, so it survives neither load nor traffic from the internet.

1.6 Mini test

1. Which flag parses a file without running it? A) `php -m file.php` B) `php -l file.php` C) `php -i file.php` D) `php -S file.php`
2. What does a PSR-style PHP file always start with? A) `<?` B) `<?php` C) `<?=` D) a shebang line only
3. True or false: the CLI and PHP-FPM always read the same `php.ini`.
4. What does the built-in server do when the requested URL matches a real file? A) it runs the file through the router anyway B) it returns that file unchanged C) it returns a 404 D) it redirects to `index.php`
5. Which statement makes scalar types strict? A) `strict_types(1);` B) `declare(strict_types=1);` C) `use strict;` D) `#[\StrictTypes]`
6. Why do PHP applications not keep state between requests? A) the web server deletes the variables B) the engine starts a fresh context for every request C) PHP variables are always static D) sessions are disabled

Answer key and explanations

1. B - `-l` (lint) parses the file and reports syntax errors only.
2. B - the full `<?php` tag works everywhere; short tags depend on a configuration switch that is off on modern servers.
3. False - each SAPI loads its own configuration file, so `php --ini` and the web server can point at different files.
4. B - static files are served directly; unknown paths go to the router or to the default 404 response.
5. B - `declare(strict_types=1)` must be the first statement of the file.
6. B - memory is reset between requests, which is why state lives in a database, a session or a cache rather than in a variable.

Chapter 2 - Project structure, coding standards and autoloading

2.1 Summary

- A conventional directory layout separates the web root from the code, the templates, the tests and the writable data.
- PSR-1 defines the basic standard: tags, class names, file names and encoding.
- PSR-12 extends it with formatting rules that tools can check automatically.
- PSR-4 maps a namespace prefix to a directory, so a class name resolves to exactly one file path.
- Autoloading means a class file is loaded the first time the class is used, not through a long list of `require` statements.
- Composer generates the autoloader for you from the `autoload` section of `composer.json`.
- A standards checker such as `phpcs` turns "the code looks wrong" into a list of concrete lines to fix.
- Following the standards costs nothing and makes pull requests about the change instead of the formatting.

2.2 Concepts

A layout that scales

output

```
my-project/
|-- public/            the only directory the web server sees
|   `-- index.php      the single entry point (front controller)
|-- src/               application code, namespace App\
|-- templates/         presentation, no business logic
|-- tests/             test code, namespace App\Tests\
|-- var/               cache, logs, sqlite files, anything writable
|-- vendor/            installed packages, never edited, never committed
|-- composer.json      dependencies and autoload rules
`-- phpunit.xml        test configuration
```

Requests enter through `public/index.php`, so no code or configuration file is ever reachable from a browser.

PSR-1 in one table

Rule	Detail
Opening tag	files use `<?php` only, never short tags
Encoding	UTF-8 without a byte order mark
Class names	`StudlyCase`, for example `LinkRepository`
One class per file	and the file name matches the class
Constants in classes	upper case with underscores
Namespaces and classes	follow one of the approved autoloading standards

PSR-12 in one table

Area	Rule
Indentation	four spaces, no tabs
Line length	soft limit of 120 characters
Braces on classes and methods	opening brace on the next line
Braces on control structures	same line as the keyword
Declarations	`declare` statements come first, one per line
Visibility	always written, `public` or `private`, never implied
Imports	one `use` per line, grouped, unused ones removed
Trailing commas	allowed in argument lists and parameters
Final newline	every file ends with exactly one
Keywords	`true`, `false` and `null` in lower case

PSR-4: from class name to file

The prefix in `composer.json` is stripped, the remaining backslashes become directory separators, and `.php` is appended:

Class	Prefix `App\` to `src/`	File
`App\Slug`	`App\` maps to `src/`	`src/Slug.php`
`App\Service\Mailer`		`src/Service/Mailer.php`
`App\Tests\SlugTest`	`App\Tests\` maps to `tests/`	`tests/SlugTest.php`

Because the mapping is deterministic, the autoloader can find any class without scanning directories.

How autoloading works

output

```
new App\Service\Mailer()
        |
        v
 engine: class unknown? -> spl_autoload_register callbacks
        |
        v
 composer autoloader: App\ -> src/, so look for
        |            src/Service/Mailer.php
        v
 require the file, then continue as if it had been
 included all along
```

`spl_autoload_register()` accepts any callable, so you can write the mechanism yourself in a few lines, as Example 3 shows.

2.3 Examples

Example 1: a file that follows PSR-1 and PSR-12.

PHP

```php
<?php

declare(strict_types=1);

namespace App\Service;

final class Slugger
{
    public function slugify(string $title): string
    {
        $slug = strtolower(trim($title));
        $slug = preg_replace('/[^a-z0-9]+/', '-', $slug);

        return trim($slug, '-');
    }
}
```

PHP

```php
<?php

declare(strict_types=1);

require __DIR__ . '/vendor/autoload.php';

use App\Service\Slugger;

$slugger = new Slugger();
echo $slugger->slugify('  Hello, PSR World! '), PHP_EOL;
```

output

```
hello-psr-world
```

Every rule visible here is mechanical: the tag, one blank line, the `declare`, the namespace, the `use`, the final newline.

Example 2: the autoload section of `composer.json`.

JSON

```json
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "App\\Tests\\": "tests/"
        }
    }
}
```

Two prefixes cannot overlap in a way that makes the resolution ambiguous, and tests use a second prefix so that test code is loaded only when it is needed.

Example 3: the autoloading mechanism, written by hand.

PHP

```php
<?php

declare(strict_types=1);

spl_autoload_register(function (string $class): void {
    $prefix = 'App\\';
    $baseDir = __DIR__ . '/src/';

    if (!str_starts_with($class, $prefix)) {
        return;
    }

    $relative = substr($class, strlen($prefix));
    $file = $baseDir . str_replace('\\', '/', $relative) . '.php';

    if (is_file($file)) {
        require $file;
    }
});

$slugger = new App\Service\Slugger();
echo $slugger->slugify('Composer does this for you'), PHP_EOL;
```

output

```
composer-does-this-for-you
```

The callback receives the fully qualified class name, turns it into a path, and requires the file if it exists. Composer registers a faster, cached version of the same idea.

Example 4: check the code against the standard.

shell

```shell
vendor/bin/phpcs --standard=PSR12 src
```

output

```
FILE: src/Service/Slugger.php
----------------------------------------------------------------------
FOUND 1 ERROR AFFECTING 1 LINE
----------------------------------------------------------------------
 14 | ERROR | Line indented incorrectly; expected 4 spaces, found 2
----------------------------------------------------------------------
```

The same violations fixed automatically look like this, with a fixer tool run before committing so reviews stay about behaviour.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
No namespace at all	every class name is global and can collide	put code in a `Vendor\Package` namespace
Two classes in one file	PSR-4 cannot resolve them	one class per file, file named after the class
Deeply nested folders per class	paths become noise	let PSR-4 derive the path from the class name
tabs mixed with spaces	inconsistent diffs, tool fights	four spaces everywhere
`require` lists at the top of every file	order breaks as the project grows	register one autoloader
Committing `vendor/`	huge diffs, version drift	commit `composer.json` and the lock file, ignore `vendor/`

- Run the standards checker in continuous integration, not only on your machine, so the rule set is the same for everybody.
- Fixers are safe on a clean working tree, because they rewrite whole files.

2.5 Practice exercises

1. A project declares the prefix `Acme\` for `src/`. Which file holds the class `Acme\Billing\InvoicePrinter`?
2. Which PSR defines the autoloading rule used by Composer?
3. Why must `public/` be the only web-visible directory?
4. Name three PSR-12 rules that a checker tool can verify mechanically.
5. What does `spl_autoload_register` accept as an argument?

Solutions

1. `src/Billing/InvoicePrinter.php` - the prefix is stripped, backslashes become directory separators, `.php` is appended.
2. PSR-4, the improved autoloading standard.
3. So that no template, test, configuration file or vendor package can be requested directly by a browser; only the front controller is the entry point.
4. Indentation width, line length limit, brace placement, visibility being always declared, and the trailing newline, among others.
5. Any callable: a closure, a static method, or an invokable object.

2.6 Mini test

1. Which encoding does PSR-1 require? A) UTF-8 with BOM B) UTF-8 without BOM C) Latin-1 D) any encoding

2. Where does PSR-12 put the opening brace of a class? A) on the same line as the class name B) on the line after the class name C) indented under the class name D) braces are forbidden for classes
3. `App\Store\Cart` maps to which file when `App\` maps to `src/`? A) `src/App/Store/Cart.php` B) `src/Store/Cart.php` C) `Store/Cart.php` D) `src/Store/Cart.class.php`
4. True or false: PSR-4 requires a fixed directory depth per namespace.
5. Which section of `composer.json` declares the PSR-4 mapping? A) `require` B) `autoload` C) `scripts` D) `config`
6. What happens when the autoloader cannot find a class file? A) PHP raises a fatal error saying the class does not exist B) PHP creates the class anyway C) PHP falls back to a global namespace version D) the file is searched for in `vendor/` only

Answer key and explanations

1. B - a byte order mark would be sent to the browser and break output.
2. B - classes and methods use the next-line brace style, while control structures keep the brace on the same line.
3. B - the prefix `App\` is replaced by `src/`, giving `src/Store/Cart.php`.
4. False - the mapping is derived from the name, so any depth is valid as long as it matches the directory tree.
5. B - the `autoload` section holds `psr-4` (and `files`, `classmap`).
6. A - the "class not found" error names the class PHP looked for, which is the first clue that a namespace or a path does not match.

Chapter 3 - Object-oriented PHP

3.1 Summary

- A class bundles state (properties) and behaviour (methods) behind a name, and `final` stops anyone from extending it by accident.
- Typed properties make the shape of an object explicit, and uninitialised typed properties throw instead of defaulting to `null`.
- Constructor property promotion declares a property and assigns the argument in one concise parameter.
- Interfaces describe behaviour, abstract classes share code, and traits copy methods into a class when neither fits.
- Enums model a fixed set of cases, and backed enums map each case to a string or an integer.
- `readonly` properties can be written once, which makes value objects safe to share and reason about.
- `match` is an expression that compares strictly and throws when no arm matches, unlike the old `switch`.

- Objects are handled by reference, so passing an object to a function does not copy it.

3.2 Concepts

The anatomy of a class

output

```
final class Order               final      cannot be extended
{
    public function __construct private readonly  per-argument
        private readonly string $id,          visibility + readonly
        private readonly int $cents,          promotion
    ) {}

    public function label(): string  return type, no implicit null
    {
        return $this->id;           $this reaches the current object
    }
}
```

Modifier	Meaning
`public`, `protected`, `private`	visibility of the member
`static`	belongs to the class, not to one instance
`readonly`	a property written exactly once, from the declaring scope
`abstract`	a method without a body, implemented by children
`final`	a class or method that cannot be overridden
`const`	a compile-time constant on the class

Types in signatures

Declaration	Accepts
`int`, `float`, `string`, `bool`	that scalar type only
`?string`	`string` or `null`
`string` or `int` together	a union of the two
`array`	any array
`iterable`	any array or `Traversable`
`object`	any object instance
`self`, `static`	the same class, `static` keeps the late-bound type
`void`	no value returned
`never`	the function never returns: it exits or throws
`mixed`	anything, useful for gradual typing

Properties may be typed the same way, but a typed property must be initialised before it is read, or PHP throws an `Error`.

Interfaces, abstract classes and traits

| | Interface | Abstract class | Trait | |-|-----------|----------------|-------| | Can be implemented/extended | many | one | any number | | Holds state | no | yes | yes | | Holds method bodies | no (constants only) | yes | yes | | Purpose | a contract | a shared base | horizontal reuse |

A class implements an interface with `implements`, extends a parent with `extends`, and pulls in traits with `use` inside the class body. One class can do all three at once.

Enums

Kind	Declared	Accessed
Pure	`enum Suit { case Hearts; }`	`Suit::Hearts`
Backed	`enum Suit: string { case Hearts = 'H'; }`	`Suit::Hearts->value`, `Suit::from('H')`

Enums are objects: they can have methods, implement interfaces, and appear in `match` arms, but they cannot hold per-instance state.

`match` versus `switch`

| | `match` | `switch` | |-|---------|----------| | Value | an expression, returns a value | a statement | | Comparison | `===`, no coercion | loose by default | | Missing case | throws `UnhandledMatchError` | silently does nothing | | Fallthrough | impossible | a classic bug source |

3.3 Examples

Example 1: a readonly value object with promoted properties.

PHP

```php
<?php

declare(strict_types=1);

final class Order
{
    public function __construct(
        private readonly string $id,
        private readonly int $cents,
    ) {
    }

    public function label(): string
    {
        return sprintf('%s costs %.2f EUR', $this->id, $this->cents / 100);
    }
}

$order = new Order(id: 'A-100', cents: 2450);
echo $order->label(), PHP_EOL;
```

output

```
A-100 costs 24.50 EUR
```

Named arguments make the call self-documenting, and `readonly` guarantees that nothing changes the amount later. Trying to write `$order->id = 'B-1'` throws an `Error`.

Example 2: a backed enum with a method and a `match`.

PHP

```php
<?php

declare(strict_types=1);

enum Status: string
{
    case Draft = 'draft';
    case Published = 'published';
    case Archived = 'archived';

    public function label(): string
    {
        return match ($this) {
            self::Draft => 'Draft',
            self::Published => 'Published',
            self::Archived => 'Archived',
        };
    }
}

$status = Status::from('published');
echo $status->value, ' => ', $status->label(), PHP_EOL;
```

output

```
published => Published
```

`Status::from()` throws a `ValueError` for an unknown word, which is exactly what you want when the value comes from a database column.

Example 3: an interface with two implementations.

PHP

```php
<?php

declare(strict_types=1);

interface Notifier
{
    public function send(string $to, string $message): void;
}

final class EmailNotifier implements Notifier
{
    public function send(string $to, string $message): void
    {
        printf('e-mail to %s: %s%s', $to, $message, PHP_EOL);
    }
}

final class SmsNotifier implements Notifier
{
    public function send(string $to, string $message): void
    {
        printf('sms to %s: %s%s', $to, $message, PHP_EOL);
    }
}

final class WelcomeService
{
    public function __construct(private Notifier $notifier)
    {
    }

    public function welcome(string $name): void
    {
        $this->notifier->send('team@example.com', 'Welcome aboard, ' . $name);
    }
}

$service = new WelcomeService(new SmsNotifier());
$service->welcome('Ada');
```

output

```
sms to team@example.com: Welcome aboard, Ada
```

The service depends on the interface, so swapping the notifier changes nothing inside it. This is the smallest useful example of the substitution principle.

Example 4: a trait that shares code between unrelated classes.

PHP

```php
<?php

declare(strict_types=1);

trait HasTimestamps
{
    public function __construct(private \DateTimeImmutable $createdAt)
    {
    }

    public function stamp(): string
    {
        return $this->createdAt->format('Y-m-d H:i');
    }
}

final class Note
{
    use HasTimestamps;
}

final class Comment
{
    use HasTimestamps;
}

$note = new Note(new \DateTimeImmutable('2026-10-10 09:30'));
$comment = new Comment(new \DateTimeImmutable('2026-10-12 18:05'));

echo $note->stamp(), PHP_EOL;
echo $comment->stamp(), PHP_EOL;
```

output

```
2026-10-10 09:30
2026-10-12 18:05
```

Two unrelated classes get the same behaviour. When several traits declare the same method, `insteadof` and `as` resolve the conflict rather than PHP guessing.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Public mutable properties	any code can break the invariants	keep properties private, expose methods
God classes with twenty dependencies	untestable and hard to name	split by responsibility
Deep inheritance chains	behaviour hides three parents away	prefer composition and interfaces
`switch` with loose comparison	`'0' == 0` surprises	use `match`, which compares strictly
Untyped properties and returns	errors surface far from the cause	type properties, parameters and returns

Mistake	Why it hurts	Best practice
Mutable value objects	two holders can disagree	`readonly` plus methods that return new instances

- Make classes `final` by default and relax only when a real extension point appears.
- Prefer small constructor argument lists: five or more arguments is a sign that one argument is itself an object you have not named yet.

3.5 Practice exercises

1. Rewrite `class Point { public $x; public $y; }` with typed properties and visibility.
2. What is the difference between `never` and `void` as a return type?
3. Why does `match` throw for an unhandled case while `switch` does not?
4. When is a trait a better choice than an abstract class?
5. What does `Status::from('nope')` do when `nope` is not a case?

Solutions

1. `final class Point { public function __construct(private readonly int $x, private readonly int $y) {} }` - typed, private, and immutable, with named construction available for free.
2. `void` means the function returns nothing but control comes back to the caller; `never` means it never returns at all, because it exits or throws.
3. `match` is designed as a safe expression: silently skipping an unknown case would hide a missing branch, so PHP raises `UnhandledMatchError`.
4. When two classes share behaviour but belong to different hierarchies, so a common parent would be wrong or impossible; traits inject the methods without a parent-child relationship.
5. It throws a `ValueError`, because the value is not backed by any case.

3.6 Mini test

1. Which keyword prevents a class from being extended? A) `static` B) `final` C) `abstract` D) `readonly`
2. How many times may a readonly property be written? A) as often as needed B) exactly once, from the declaring scope C) only in the constructor of a child class D) never
3. What does an uninitialised typed property do when read? A) returns `null` B) returns an empty string C) throws an `Error` D) returns `0`
4. Which comparison does `match` use? A) `==` B) `===` C) `in_array` with loose flags D) it depends on the case type
5. True or false: a trait may declare properties and a constructor.
6. What happens when a class implements an interface but misses a method? A) nothing until the method is called B) PHP raises a fatal error when the class is declared C) the method returns `null` automatically D) PHP fills in an empty body

Answer key and explanations

1. B - `final` on the class blocks inheritance; on a method it blocks overriding.
2. B - the write happens during initialisation, typically from the constructor, and any later assignment throws.
3. C - typed properties are uninitialised until assigned, and reading them raises an `Error` naming the property and class.
4. B - `match` arms are compared with `===`, so no scalar coercion or type juggling occurs.
5. True - traits are compiled into the class, so they can add properties, constructors and abstract method signatures.
6. B - a class that does not satisfy the interface is a fatal error at declaration time, which is why running the linter early helps.

Chapter 4 - Namespaces, the SPL and CLI scripting

4.1 Summary

- A namespace qualifies a name, so `App\Cli` and `Vendor\Cli` can both define a `Runner` without colliding.
- `use` imports a name into the current file and may alias it when two names clash.
- Names are resolved differently depending on whether they are unqualified, qualified or fully qualified.
- The Standard PHP Library (SPL) ships data structures such as `SplStack`, `SplQueue` and `ArrayObject`, plus iterator interfaces.
- A generator function uses `yield` to produce values lazily, which keeps memory flat for large inputs.
- Command-line scripts read `$argv`, write to `STDOUT` and `STDERR`, and end with an exit code the shell can inspect.
- `getopt()` parses short and long options without pulling in a library.
- Xdebug provides step debugging, richer stack traces and coverage; its modes are switched in `php.ini` or per run with `php -d`.

4.2 Concepts

Namespaces in one table

Form	Written	Resolves to
Unqualified name	`new Runner()`	current namespace
Qualified name	`new Http\Client()`	current namespace plus the name

Form	Written	Resolves to
Fully qualified name	`new \DateTimeImmutable()`	from the global root
Import	`use App\Model\Link;`	alias for the file, no runtime cost
Import with alias	`use App\Model\Link as L;`	choose a local name

Every namespace maps to a directory through PSR-4, so the namespace is also a hint about where the file lives.

Name resolution in practice

output

```
file declares:  namespace App\Cli;

use App\Model\Link;          Link         -> App\Model\Link
use App\Model\Link as L;      L           -> App\Model\Link
                               Client      -> App\Cli\Client
                               \Client     -> Client (global)
str_starts_with(...)          built-ins stay reachable at any time
```

Inside a namespace, unqualified function and constant names fall back to the global set, but unqualified class names do not: `new DateTimeImmutable()` fails inside a namespace unless you write `new \DateTimeImmutable()` or import it.

SPL structures worth knowing

Structure	Behaviour	Typical job
`SplStack`	last in, first out	undo history
`SplQueue`	first in, first out	job queue
`SplDoublyLinkedList`	both ends	playlists, work lists
`ArrayObject`	an array wrapped in an object	passing collections around
`SplFixedArray`	fixed size, integer keys	memory-sensitive buffers
`IteratorAggregate`, `Iterator`	make your own class iterable	custom collections

Anything implementing `Traversable` can be used in `foreach`, and `iterator_to_array()` converts it into a plain array when needed.

Generators

output

```
function rows(): Generator
{
    yield 1;            pauses here, returns 1
    yield 2;            resumes on the next request
}

foreach (rows() as $row)   asks for one value at a time
```

A generator keeps its own state between resumes, so reading a large file line by line costs one line of memory instead of the whole file.

The CLI environment

Item	Meaning
`$argv[0]`	the script name
`$argv[1..]`	the arguments after the script name
`$argc`	the number of arguments, including the script
`STDIN`, `STDOUT`, `STDERR`	predefined streams
`exit(0)`, `exit(1)`	success, failure; other codes are allowed
`getopt('f:', ['file:'])`	parse `-f value` and `--file value`
`PHP_EOL`	the line ending of the current platform

A command that fails should write a message to `STDERR` and exit with a non-zero code, so that shell pipelines and CI jobs can react.

Xdebug modes

Mode	What it does
`off`	nothing, the safe default in production
`develop`	nicer stack traces and `var_dump` output
`debug`	step debugging with an IDE key
`coverage`	collects line coverage for tests
`trace`	writes every function call to a file

Modes combine: `xdebug.mode=develop,coverage` turns both on for one run.

4.3 Examples

Example 1: namespaces and imports across two files.

PHP

```php
<?php

declare(strict_types=1);

namespace App\Model;

final class Link
{
    public function __construct(
        public readonly string $title,
        public readonly string $url,
    ) {
    }
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace App\Cli;

use App\Model\Link;

$link = new Link('PHP manual', 'https://www.php.net/manual/en/');
echo $link->title, ' -> ', $link->url, PHP_EOL;
```

output

```
PHP manual -> https://www.php.net/manual/en/
```

The `use` statement is a compile-time alias: it changes how the name in this file is read and costs nothing at runtime.

Example 2: a generator over a long input.

PHP

```php
<?php

declare(strict_types=1);

function lines(string $text): \Generator
{
    foreach (explode("\n", trim($text)) as $line) {
        yield trim($line);
    }
}

foreach (lines("ada\n grace \n linus") as $i => $line) {
    echo $i, ': ', $line, PHP_EOL;
}
```

output

```
0: ada
1: grace
2: linus
```

Each `yield` suspends the function and hands one value back, so the loop body runs before the next line is produced.

Example 3: SPL stacks and queues.

PHP

```php
<?php

declare(strict_types=1);

$undo = new SplStack();
$undo->push('type a');
$undo->push('type b');
echo $undo->pop(), PHP_EOL;
echo $undo->count(), ' event left', PHP_EOL;

$jobs = new SplQueue();
$jobs->enqueue('job-1');
$jobs->enqueue('job-2');
echo $jobs->dequeue(), PHP_EOL;
```

output

```
type b
1 event left
job-1
```

The stack returns the last value pushed; the queue returns the first value enqueued. Both are O(1) because they are linked lists internally.

Example 4: a command-line script that validates its input.

PHP

```php
<?php

declare(strict_types=1);

if ($argc < 2) {
    fwrite(STDERR, 'usage: add.php <url>', PHP_EOL);
    exit(1);
}

$url = $argv[1];

if (filter_var($url, FILTER_VALIDATE_URL) === false) {
    fwrite(STDERR, 'invalid url: ' . $url, PHP_EOL);
    exit(2);
}

echo 'accepted: ', $url, PHP_EOL;
```

output

```
accepted: https://getcomposer.org
```

Run with no argument it prints `usage: add.php <url>` to standard error and exits with code 1, which a shell script can test with `if ! php add.php`.

Example 5: run PHP with Xdebug in develop mode.

shell

```
php -d xdebug.mode=develop debug.php
```

output

```
/Users/ada/projects/debug.php:6:
array(2) {
  [0] => string(3) "red"
  [1] => string(5) "green"
}
```

The enabling line in `php.ini` looks like this:

INI

```
xdebug.mode=develop,coverage
xdebug.start_with_request=no
```

Coverage mode is what lets a coverage report tell you which lines a test never reached; develop mode only improves the human-readable output.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
One class per file, but no namespace	names collide with packagist packages	namespace every project `Vendor\Package`
Mixing fully qualified and imported names randomly	readers cannot tell where a class comes from	import at the top, alias only on clash
Catching nothing in a CLI script	failures look like success to CI	write to `STDERR` and exit non-zero
Long argument parsing by hand	off-by-one errors with options and values	use `getopt()` or a dedicated component
Leaving Xdebug enabled in production	slow requests, information disclosure	`xdebug.mode=off` on live servers
Reaching for a framework for a 30-line script	dependencies you cannot audit	the SPL plus a small autoloader is often enough

- Keep CLI code in its own namespace, so console entry points are easy to find.
- Streams are resources: closing them explicitly keeps long-running scripts from leaking file handles.

4.5 Practice exercises

1. Inside `namespace App\Cli;`, what must you write to construct the global `DateTimeImmutable` class?
2. Which SPL structure would you use for an undo history, and why?
3. What does a generator return when the caller asks for a value?

4. Which stream should an error message go to, and why?
5. How do you turn on Xdebug for a single run without editing `php.ini`?

Solutions

1. Either `new \DateTimeImmutable()` or import it with `use DateTimeImmutable;` at the top of the file, because class names are not resolved against the global namespace inside a namespace.
2. `SplStack`, because a stack pops the most recent item first, which is the order an undo action needs.
3. The function pauses at `yield` and produces one value at a time; the caller resumes it for the next value, which keeps memory usage low.
4. Standard error, so that normal output can still be piped into another program while failures remain visible.
5. `php -d xdebug.mode=develop script.php` sets the directive for that run only.

4.6 Mini test

1. Which statement imports a class under a different local name? A) `use App\Model\Link as L;` B) `import App\Model\Link;` C) `require App\Model\Link as L;` D) `namespace L = App\Model\Link;`
2. What does `$argv[0]` contain? A) the first argument B) the script name C) the number of arguments D) the current working directory
3. Which keyword turns a function into a generator? A) `yield` B) `defer` C) `await` D) `generate`
4. True or false: `SplQueue` returns the last enqueued element first.
5. Which Xdebug mode produces code coverage data? A) `develop` B) `coverage` C) `trace` D) `profile`
6. Inside a namespace, which unqualified name falls back to the global namespace? A) class names B) functions and constants C) interfaces only D) nothing falls back

Answer key and explanations

1. A - `use ... as ...` creates an alias local to the file.
2. B - the script itself is element zero; real arguments start at one.
3. A - a `yield` inside the body makes the function return a generator.
4. False - a queue is first in, first out; the stack pops the newest item.
5. B - `coverage` collects the executed lines; `develop` only improves human-readable output and `trace` writes call logs.
6. B - functions and constants fall back to the global set when they are not found in the current namespace, while classes do not.

Chapter 5 - Composer and dependency management

5.1 Summary

- Composer is the dependency manager for PHP: one `composer.json` per project declares packages, autoload rules and project scripts.
- Packagist is the public package registry, and packages are versioned with semantic version numbers.
- `composer install` installs exactly what the lock file pins, while `composer update` moves packages inside their allowed ranges.
- The lock file belongs in version control; the `vendor/` directory does not, because it can always be rebuilt.
- Autoloading is configured in the same file, so installing a package and using its classes are the same step.
- Scripts give every contributor the same short commands for linting, testing and cache clearing.
- A version constraint describes the range you accept, not the single version you happened to test.
- `composer validate` checks the manifest, and `composer audit` reports known security advisories in the locked versions.

5.2 Concepts

The manifest, section by section

Field	What it holds
`name`	`vendor/package`, unique on Packagist
`description`, `type`, `license`	metadata for humans and tools
`require`	runtime packages, plus `php` and `ext-*` requirements
`require-dev`	packages needed only while developing
`autoload`	PSR-4 prefixes and extra files loaded on every request
`autoload-dev`	the same, for the test namespace only
`scripts`	named shortcuts and event hooks
`config`	project defaults such as `sort-packages`
`repositories`	where to look when the default registry is not enough

The `ext-*` entries matter on servers: they fail the install early when a PHP extension is missing, instead of producing a confusing runtime error.

Version constraints

Constraint	Range accepted
`3.6.0`	exactly `3.6.0`
`^3.6`	`>=3.6.0` and `<4.0.0`
`~3.6.0`	`>=3.6.0` and `<3.7.0`
`3.*`	any `3.x` release
`>=3.5 <4`	an explicit range
`*`	anything, rarely what you want
`dev-main`	a branch, useful only while contributing upstream

The caret is the common choice: it accepts compatible releases and stops before the next major version. Constraint the `php` entry too, so nobody installs the project on an interpreter it has never seen.

The commands you will actually type

Command	Job
`composer init`	answer questions, write a first `composer.json`
`composer install`	install the locked versions, creating `vendor/`
`composer update`	move packages inside their constraints, rewrite the lock
`composer require vendor/pkg`	add a dependency and install it
`composer require --dev vendor/pkg`	add a development dependency
`composer remove vendor/pkg`	remove a dependency and its autoload entry
`composer dump-autoload -o`	rebuild the autoloader, optimised
`composer run-script test`	run a script; `composer test` is the shorthand
`composer validate`	check that the manifest is valid
`composer audit`	report advisories for the locked versions
`composer outdated`	show packages with newer releases available

Lock file, vendor, and reproducibility

output

```
composer.json   composer.lock        vendor/
what I accept    what CI installed     what PHP loads
```

`composer install` reads the lock, so two developers and the server all run the same code. `composer update` rewrites the lock, which is why it deserves its own commit and its own review. The lock file also records the hashes Composer used, and an updated `composer.json` without a matching lock makes `install` refuse to run until you update it.

Scripts

Scripts are named commands executed by Composer. A script name starting with `@` calls another script, and a list runs in order. Packages may also hook into events such as `post-install-cmd`, which is how tools warm caches after an install.

5.3 Examples

Example 1: a complete small `composer.json`.

JSON

```json
{
    "name": "acme/links",
    "description": "A small links manager",
    "type": "project",
    "license": "MIT",
    "require": {
        "php": ">=8.3",
        "ext-pdo_sqlite": "*",
        "monolog/monolog": "^3.6"
    },
    "require-dev": {
        "phpunit/phpunit": "^10.5",
        "squizlabs/php_codesniffer": "^3.10"
    },
    "autoload": {
        "psr-4": {
            "Acme\\": "src/"
        },
        "files": [
            "src/functions.php"
        ]
    },
    "autoload-dev": {
        "psr-4": {
            "Acme\\Tests\\": "tests/"
        }
    },
    "scripts": {
        "lint": "phpcs --standard=PSR12 src tests",
        "test": "phpunit",
        "check": ["@lint", "@test"]
    },
    "config": {
        "sort-packages": true
    }
}
```

The `files` entry loads one plain file of helper functions on every request, which is the one place where an autoloader cannot help.

Example 2: add a package.

shell

```
composer require monolog/monolog
```

output

```
Using version ^3.6 for monolog/monolog
./composer.json has been updated (2 changes)
Running composer update monolog/monolog
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking monolog/monolog (3.6.0)
Writing lock file
Installing dependencies from lock file
Package operations: 1 install, 0 updates, 0 removals
  - Installing monolog/monolog (3.6.0): Extracting archive
Generating autoload files
```

The command picks the newest compatible version, adds it to `require`, locks it, installs it, and regenerates the autoloader in one go.

Example 3: use the installed package.

PHP

```php
<?php

declare(strict_types=1);

require __DIR__ . '/vendor/autoload.php';

use Monolog\Logger;
use Monolog\Handler\StreamHandler;

$log = new Logger('app');
$log->pushHandler(new StreamHandler(__DIR__ . '/var/app.log', Logger::WARNING));

$log->warning('Disk almost full', ['free' => '2 GB']);
$log->info('this line stays out of the file');

echo file_get_contents(__DIR__ . '/var/app.log');
```

output

```
[2026-10-10T09:30:00.123456+00:00] app.WARNING: Disk almost full {"free":"2 GB"} []
```

The handler level filters the record: `warning` passes, `info` does not. The timestamp is the moment you run the script, and the context array is encoded as JSON at the end of the line.

Example 4: project scripts.

shell

```
composer check
```

output

```
> phpcs --standard=PSR12 src tests
> phpunit
```

Every contributor runs the same checks with the same flags, and a continuous integration job needs one command instead of three.

Example 5: part of a lock file.

JSON

```
{
    "packages": [
        {
            "name": "monolog/monolog",
            "version": "3.6.0",
            "require": {
                "php": ">=8.1",
                "psr/log": "^2.0 || ^3.0"
            }
        }
    ]
}
```

Each locked package records the exact version and its own requirements, which is how Composer computes a consistent set without asking the network twice.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Committing `vendor/`	huge diffs, review noise, drift	ignore it, commit the lock file
Running `composer update` before every deploy	production changes without review	`composer install` on servers, `update` in a reviewed branch
Pinning `*` or a loose `~` on old majors	surprise upgrades break the app	use `^` and read the changelog on majors
Editing files inside `vendor/`	the next install erases the work	extend or decorate the package in your own code
Forgetting `ext-*` requirements	the app passes locally and dies on the server	declare every extension you use
Running Composer as root inside containers	silently unsafe permissions	set `COMPOSER_ALLOW_SUPERUSER` deliberately or use a build stage

- Run `composer validate --strict` in continuous integration to catch typos in the manifest before they reach a release.
- Read `composer audit` output as part of dependency review, not only when an incident happens.

5.5 Practice exercises

1. What is the difference between `composer install` and `composer update`?
2. Which two files should be committed, and which directory should not?
3. What does the constraint `^2.4` accept for the `monolog/monolog` version?
4. How do you add a package that only the tests need?
5. What is the effect of `composer dump-autoload -o`?

Solutions

1. `install` installs what `composer.lock` pins and is reproducible; `update` recalculates versions inside the constraints declared in `composer.json` and rewrites the lock file.
2. `composer.json` and `composer.lock`; `vendor/` stays out of version control.
3. Any `2.x` release greater than or equal to `2.4`, so `2.4`, `2.9` and even `2.99`, but not `3.0`.
4. `composer require --dev vendor/package`.
5. It rebuilds the autoloader as an optimised class map, which is faster in production but must be regenerated whenever classes are added or moved.

5.6 Mini test

1. Which file records the exact installed versions? A) `composer.json` B) `composer.lock` C) `vendor/installed.json` D) `autoload.json`
2. What does `^1.4` mean? A) exactly `1.4` B) `>=1.4` and `<2.0` C) `>=1.4` and `<1.5` D) any `1.x`
3. Which command adds a package needed only during development? A) `composer install --dev pkg` B) `composer require --dev pkg` C) `composer update --dev pkg` D) `composer add pkg`
4. True or false: `vendor/` should be committed so the server can skip installing.
5. Where are PSR-4 prefixes declared? A) in the `require` section B) in the `autoload` section C) in the `scripts` section D) in a separate `autoload.json`
6. Which command reports security advisories for installed packages? A) `composer audit` B) `composer check` C) `composer security` D) `composer outdated --security`

Answer key and explanations

1. B - the lock file holds the resolved versions and hashes for every dependency.
2. B - the caret allows compatible releases: same major version, at or above the stated minor.
3. B - `--dev` puts the package in `require-dev`, so it is installed for development and skipped by `--no-dev` deployments.
4. False - the directory is generated; keeping it in version control causes huge diffs and hides which lock entry is actually in use.
5. B - `autoload.psr-4` maps prefixes to directories.
6. A - `composer audit` checks the locked versions against the public advisory database.

Chapter 6 - Design patterns and dependency injection

6.1 Summary

- A design pattern is a named, well-understood solution to a recurring structural problem.
- The vocabulary is the point: "a repository" and "a strategy" communicate in two words what would otherwise take a paragraph.

- A repository hides storage behind an interface, so domain code never writes SQL and storage can change underneath it.
- A strategy swaps behaviour by giving one job several interchangeable implementations behind an interface.
- A factory centralises construction when creating an object requires a decision rather than a `new` expression.
- Dependency injection means a class receives its collaborators instead of building them itself.
- A container is a map from interface to factory closure; it is a convenience, not a requirement.
- Patterns used before they are needed cost more than the problem they would have solved.

6.2 Concepts

Four patterns, in the order they appear in real code

Pattern	Problem	Shape in PHP
Repository	storage leaks into the domain	an interface plus a PDO implementation
Strategy	behaviour varies by context	several classes implementing one interface
Factory	construction needs a decision	a static or injected creator returning an interface
Dependency injection	collaborators hard-coded in constructors	parameters typed as interfaces, wired at the edge

Repository

output

```
LinkService                LinkRepository (interface)
    |  all(): array  ---->  all(): array
    |  save(Link)           save(Link)
                                ^
                                | implements
                          PdoLinkRepository  (the only class that knows SQL)
```

The service depends on the interface, so a test can supply an array-backed implementation and a deployment can switch from SQLite to MySQL without touching the service.

Strategy

output

```
Report --> Presenter (interface)
              |-> TablePresenter   renders an ASCII table
              `-> JsonPresenter    renders JSON for an API
```

The caller chooses the implementation once, and the rendering code stays in exactly one place per strategy.

Dependency injection, by hand

Style	Written	When it fits
Constructor	parameter in `__construct`	always the first choice
Setter	`setLogger(Logger $l)`	optional collaborators, circular graphs
Interface	`setContainerAware(...)`	frameworks that must stay generic

Constructor injection makes a dependency required and visible, which is why it is preferred over the alternatives.

Containers

A container is nothing more than a map from a name to a closure that builds the object:

output

```
$container['mailer'] = fn () => new SmtpMailer(...);
$container['report']  = fn ($c) => new Report($c['mailer']);
```

Resolution is lazy: nothing is built until someone asks. A container is worth writing only when the same object graph is assembled in more than one place, which in a small application means exactly one front controller.

6.3 Examples

Example 1: a repository interface and its PDO implementation.

PHP

```php
<?php

declare(strict_types=1);

namespace App\Repository;

use App\Entity\Link;

interface LinkRepository
{
    /** @return list<Link> */
    public function all(): array;

    public function save(Link $link): void;
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace App\Repository;

use App\Entity\Link;
use PDO;

final class PdoLinkRepository implements LinkRepository
{
    public function __construct(private PDO $connection)
    {
    }

    public function all(): array
    {
        $rows = $this->connection
            ->query('SELECT id, title, url, category FROM links ORDER BY id DESC')
            ->fetchAll(PDO::FETCH_ASSOC);

        return array_map(
            static fn (array $row): Link => new Link(
                $row['title'],
                $row['url'],
                Category::from($row['category']),
                (int) $row['id'],
            ),
            $rows,
        );
    }

    public function save(Link $link): void
    {
        $statement = $this->connection->prepare(
            'INSERT INTO links (title, url, category) VALUES (?, ?, ?)'
        );
        $statement->execute([
            $link->title,
            $link->url,
            $link->category->value,
        ]);
    }
}
```

Two classes, one interface, and the SQL lives in a single place.

Example 2: a strategy chosen by the caller.

PHP

```php
<?php

declare(strict_types=1);

interface Presenter
{
    public function render(array $rows): string;
}

final class LinePresenter implements Presenter
{
    public function render(array $rows): string
    {
        return implode("\n", array_map(
            static fn (array $row): string => sprintf('- %s', $row['title']),
            $rows,
        ));
    }
}

final class JsonPresenter implements Presenter
{
    public function render(array $rows): string
    {
        return json_encode($rows, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
    }
}

$rows = [['title' => 'PHP manual'], ['title' => 'Composer docs']];
echo (new JsonPresenter())->render($rows), PHP_EOL;
```

output

```
[
    {
        "title": "PHP manual"
    },
    {
        "title": "Composer docs"
    }
]
```

`JSON_PRETTY_PRINT` indents with four spaces, which is why the output has the shape above.

Example 3: a factory that decides what to build.

PHP

```php
<?php

declare(strict_types=1);

interface Logger
{
    public function write(string $message): void;
}

final class FileLogger implements Logger
{
    public function write(string $message): void
    {
        file_put_contents(__DIR__ . '/var/app.log', $message . PHP_EOL, FILE_APPEND);
    }
}

final class NullLogger implements Logger
{
    public function write(string $message): void
    {
    }
}

final class LoggerFactory
{
    public static function make(string $environment): Logger
    {
        return match ($environment) {
            'prod' => new FileLogger(),
            default => new NullLogger(),
        };
    }
}

$logger = LoggerFactory::make('dev');
$logger->write('nothing is written in development');
```

output

```
(no output: the null logger discards the message)
```

Switching the environment string changes the behaviour of every call site without any call site changing.

Example 4: a five-line container.

PHP

```php
<?php

declare(strict_types=1);

final class Container
{
    /** @var array<string, callable> */
    private array $factories = [];

    private array $built = [];

    public function set(string $id, callable $factory): void
    {
        $this->factories[$id] = $factory;
    }

    public function get(string $id): mixed
    {
        if (!isset($this->built[$id])) {
            $this->built[$id] = ($this->factories[$id])($this);
        }

        return $this->built[$id];
    }
}

$container = new Container();
$container->set('greeting', static fn (): string => 'hello from the container');

echo $container->get('greeting'), PHP_EOL;
```

output

```
hello from the container
```

Each closure receives the container, so a service can ask for its own dependencies by name and the graph is built on demand.

Example 5: why injection matters for tests.

PHP

```php
<?php

declare(strict_types=1);

namespace App;

final class LinkCounter
{
    public function __construct(private LinkRepository $links)
    {
    }

    public function count(): int
    {
        return count($this->links->all());
    }
}
```

A test passes a fake repository that returns two links, and `count()` returns two without touching a database, a network or a file.

output

```
2
```

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
A repository interface per table	dozens of empty interfaces	start with a concrete class, extract when needed
Service locator called from everywhere	dependencies disappear from signatures	inject interfaces through constructors
A container in a class that does not need it	hidden couplings, harder testing	build the graph in the entry point only
Singletons for shared state	global mutable state, untestable	pass the object explicitly
Patterns applied to a 200-line script	more code than the problem	plain functions and arrays until pain appears
Interfaces with a single forever implementation	indirection with no benefit	let the second implementation justify the interface

- Name the collaborator in the parameter type, never read it from a global registry inside a method body.
- If you cannot write a fake for a class in under ten lines, its dependency list is too long.

6.5 Practice exercises

1. Which class in a repository pair is allowed to contain SQL?
2. Why does a service depend on `LinkRepository` rather than `PdoLinkRepository`?
3. A report must print as HTML for the browser and as JSON for the API. Which pattern fits, and what do you implement?
4. What does a container give you that a bunch of `new` expressions does not?
5. When should you not introduce a repository interface?

Solutions

1. Only the implementation, for example `PdoLinkRepository`; the interface names operations, never storage.
2. Because the service only needs the operations; depending on the concrete class would couple it to PDO and make tests require a live database.
3. The strategy pattern: define a `Presenter` interface with a `render()` method and implement `HtmlPresenter` and `JsonPresenter`.

4. Lazy, named construction with a single place where the object graph is assembled, so wiring code is written once instead of at every call site.
5. When there is only one implementation and no second one on the horizon: an interface then adds indirection without buying substitution.

6.6 Mini test

1. What does a repository hide? A) the domain rules B) the storage mechanism C) the HTTP layer D) the template engine
2. Which injection style is preferred by default? A) property injection B) constructor injection C) a global registry D) `func_get_args()`
3. What does a strategy pattern require? A) a base class with a template method B) several classes implementing one interface C) a singleton D) static factory methods only
4. True or false: a container must be a large library.
5. What is a factory responsible for? A) caching objects B) deciding which object to create and returning it C) mapping routes to controllers D) escaping output
6. Which is a symptom of over-engineering? A) one interface with two implementations B) an interface per class with only one implementation each C) a named constructor `fromArray()` D) a value object with readonly properties

Answer key and explanations

1. B - callers ask for data; they never learn whether it came from SQLite, MySQL or an array.
2. B - a required constructor argument is explicit, immutable for the life of the object, and obvious in the signature.
3. B - interchangeable implementations behind one interface, chosen by the caller or by configuration.
4. False - a container can be a ten-line class holding closures; scale it only when the project needs it.
5. B - a factory turns a decision into a single place that returns an interface, so the decision is not repeated at every call site.
6. B - one-implementation interfaces add a file and an indirection without buying substitutability.

Chapter 7 - Databases with PDO

7.1 Summary

- PDO is one object-oriented interface that talks to many databases through interchangeable drivers.
- A DSN names the driver and the target, and constructor options tune the connection.

- Setting `PDO::ATTR_ERRMODE` to `PDO::ERRMODE_EXCEPTION` turns every database error into a thrown exception instead of a silent failure.
- Prepared statements keep the SQL text and the values apart, which is what makes SQL injection impossible.
- Named placeholders make statements with many values readable, while positional placeholders are shorter.
- Fetch modes decide the shape of a returned row: array, object, single column or a populated class instance.
- Transactions group statements so that a failure leaves the database exactly as it was.
- A connection belongs to one request; sharing it between requests through a session or a static property breaks sooner or later.

7.2 Concepts

Connecting

Driver	DSN example	Notes
SQLite	`sqlite:/path/to/file.db`	also `sqlite::memory:` for tests
MySQL	`mysql:host=localhost;dbname=app;charset=utf8mb4`	always set the charset
PostgreSQL	`pgsql:host=localhost;dbname=app`	set `sslmode` in production

PHP

```php
$pdo = new PDO($dsn, $user, $password);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Extensions matter here: `pdo_sqlite` is bundled with most builds of PHP, while MySQL and PostgreSQL drivers come from the matching extension.

Attributes worth setting

Attribute	Value	Effect
`ATTR_ERRMODE`	`ERRMODE_EXCEPTION`	errors throw `PDOException`
`ATTR_DEFAULT_FETCH_MODE`	`FETCH_ASSOC`	every fetch returns an associative array
`ATTR_EMULATE_PREPARES`	`false`	use real server-side prepares where available
`ATTR_STRINGIFY_FETCHES`	`false`	keep integers as integers

Turning off emulated prepares on MySQL, in particular, makes the server do the parameter binding instead of the driver doing it in PHP.

Fetch modes

Mode	A row looks like
FETCH_ASSOC	['id' => 1, 'title' => 'PHP']
FETCH_NUM	[0 => 1, 1 => 'PHP']
FETCH_BOTH	both of the above at once
FETCH_OBJ	an anonymous object with $row->title
FETCH_COLUMN	a single column, for counts and lists
FETCH_CLASS	an instance of a class you name

Why prepared statements are not optional

output

```
$sql = 'SELECT * FROM users WHERE name = ' . $_GET['name'];

input:  ' OR 1 = 1 --
becomes: SELECT * FROM users WHERE name = ' OR 1 = 1 --   the whole table!

$statement = $pdo->prepare('SELECT * FROM users WHERE name = ?');
$statement->execute([$_GET['name']]);     the value never becomes SQL
```

With a prepared statement the database parses the query first and receives the value afterwards, as a value: no quoting trick can turn data into code.

Transactions

output

```
beginTransaction()
   |
   +-- INSERT ...        \
   +-- UPDATE ...         >  either all of them commit,
   +-- DELETE ...        /   or all of them roll back
   |
commit()  ... or ... rollBack()
```

Nested work uses SAVEPOINT, and a caught exception should always lead to `rollBack()` before the connection is reused.

7.3 Examples

Example 1: connect to SQLite and create a table.

PHP

```php
<?php

declare(strict_types=1);

$pdo = new PDO('sqlite:' . __DIR__ . '/var/links.db');
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$pdo->exec(
    'CREATE TABLE IF NOT EXISTS links (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        url TEXT NOT NULL,
        category TEXT NOT NULL
    )'
);

echo 'table ready', PHP_EOL;
```

output

```
table ready
```

`exec()` is for statements that return no rows, such as DDL or a bulk update.

Example 2: insert with named placeholders, twice, and read the new id.

PHP

```php
<?php

declare(strict_types=1);

$insert = $pdo->prepare(
    'INSERT INTO links (title, url, category) VALUES (:title, :url, :category)'
);

$insert->execute([
    'title' => 'PHP manual',
    'url' => 'https://www.php.net',
    'category' => 'docs',
]);
$insert->execute([
    'title' => 'Composer',
    'url' => 'https://getcomposer.org',
    'category' => 'docs',
]);

echo 'last id: ', $pdo->lastInsertId(), PHP_EOL;
```

output

```
last id: 2
```

Preparing once and executing many times is both faster and safer than building a new query string per row.

Example 3: read rows back.

PHP

```php
<?php

declare(strict_types=1);

$rows = $pdo
    ->query('SELECT id, title, url FROM links ORDER BY id')
    ->fetchAll(PDO::FETCH_ASSOC);

foreach ($rows as $row) {
    printf('%d %s %s%s', $row['id'], $row['title'], $row['url'], PHP_EOL);
}

echo count($rows), ' link(s)', PHP_EOL;
```

output

```
1 PHP manual https://www.php.net
2 Composer https://getcomposer.org
2 link(s)
```

`fetchAll()` returns an array of rows; for very large result sets iterate over the statement object itself so that one row is held at a time.

Example 4: update, count affected rows, delete.

PHP

```php
<?php

declare(strict_types=1);

$update = $pdo->prepare('UPDATE links SET title = :title WHERE id = :id');
$update->execute(['title' => 'PHP documentation', 'id' => 1]);
echo 'updated rows: ', $update->rowCount(), PHP_EOL;

$delete = $pdo->prepare('DELETE FROM links WHERE id = :id');
$delete->execute(['id' => 2]);
echo 'deleted rows: ', $delete->rowCount(), PHP_EOL;

echo $pdo->query('SELECT COUNT(*) FROM links')->fetchColumn(), ' left', PHP_EOL;
```

output

```
updated rows: 1
deleted rows: 1
1 left
```

`rowCount()` reports how many rows a statement changed, and `fetchColumn()` pulls a single scalar, which is the cheapest way to count.

Example 5: a transaction that rolls back.

PHP

```php
<?php

declare(strict_types=1);

function renameLink(PDO $pdo, int $id, string $title, bool $commit): void
{
    try {
        $pdo->beginTransaction();

        $statement = $pdo->prepare('UPDATE links SET title = :title WHERE id = :id');
        $statement->execute(['title' => $title, 'id' => $id]);

        if ($commit === false) {
            throw new RuntimeException('not approved yet');
        }

        $pdo->commit();
        echo 'committed', PHP_EOL;
    } catch (Throwable $e) {
        $pdo->rollBack();
        echo 'rolled back: ', $e->getMessage(), PHP_EOL;
    }
}

renameLink($pdo, 1, 'Approved title', false);
echo $pdo->query('SELECT title FROM links WHERE id = 1')->fetchColumn(), PHP_EOL;
```

output

```
rolled back: not approved yet
PHP documentation
```

The update was never committed, so the original title is still there. `commit()` stays inside the `try` and `rollBack()` in the `catch`, which keeps the two sides symmetric.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Interpolating variables into SQL	SQL injection, the classic breach	prepared statements with placeholders
Leaving `ERRMODE_SILENT` in place	failed queries return `false` and vanish	use `ERRMODE_EXCEPTION` and catch `PDOException`
Building `WHERE id IN (...)` by string	both unsafe and slow in loops	prepare a statement per row, or bind a placeholder set
`SELECT *` in application code	breaks when the table gains a column	name the columns you use
Catching an exception and continuing a transaction	the connection stays in a transaction	roll back in the `catch`, then decide
Storing the connection in a session or static	connection errors leak across requests	open one connection per request

- Keep SQL in the repository layer, so the rest of the application names methods such as `all()` and `save()` instead of tables and columns.
- For schema changes use migration files, applied in order, rather than ad-hoc `CREATE TABLE` statements in application code.

7.5 Practice exercises

1. Write the `prepare`/`execute` pair that inserts a link with named placeholders.
2. Which attribute turns PDO errors into exceptions, and what is its value?
3. What does `rowCount()` report after an `UPDATE` that matched three rows?
4. Why is `SELECT *` a poor habit in application code?
5. What must happen in the `catch` block of a failed transaction?

Solutions

1. `$statement = $pdo->prepare('INSERT INTO links (title, url, category) VALUES (:title, :url, :category)');` followed by `$statement->execute(['title' => $t, 'url' => $u, 'category' => $c]);`
2. `PDO::ATTR_ERRMODE` with the value `PDO::ERRMODE_EXCEPTION`, set through `setAttribute()` right after connecting.
3. The number of rows actually changed, so three in this case.
4. Because the result set changes shape when a column is added or reordered, and because it transfers data the application never reads.
5. A `rollBack()`, so the connection is usable again and no half-applied change survives.

7.6 Mini test

1. Which DSN opens a SQLite database file? A) `sqlite://path` B) `sqlite:/path/to/file.db` C) `sqlite(path/to/file.db)` D) `pdo:sqlite:/path`
2. What does `ERRMODE_EXCEPTION` change? A) it makes queries faster B) errors are thrown as `PDOException` C) it disables transactions D) it logs every query automatically
3. Which fetch mode returns rows as associative arrays? A) `FETCH_NUM` B) `FETCH_OBJ` C) `FETCH_ASSOC` D) `FETCH_CLASS`
4. True or false: a prepared statement can be executed again with new values after being prepared once.
5. What does `lastInsertId()` return? A) the number of inserted rows B) the id generated by the last insert on that connection C) the highest id in the table D) the id of the last prepared statement
6. Which statement about transactions is correct? A) they make queries faster B) a rollback undoes every statement since the begin C) they require `ERRMODE_SILENT` D) an exception inside them commits automatically

Answer key and explanations

1. B - the DSN is the driver prefix `sqlite:` followed by the file path; `sqlite::memory:` gives a database that lives in RAM.
2. B - errors become exceptions, so a failed query cannot be mistaken for an empty result.
3. C - the associative mode maps column names to values.
4. True - prepare once, then call `execute()` with a different array each time, which is why it is the recommended pattern.
5. B - it reports the identifier the database generated for the most recent `INSERT` on that connection.
6. B - statements issued after `beginTransaction()` are discarded by `rollBack()`, which is what keeps related changes atomic.

Chapter 8 - Templating, escaping, UTF-8 and dates

8.1 Summary

- A template separates the shape of a page from the code that decides what goes in it, so a designer can edit one file without breaking logic.
- Plain PHP templates are ordinary PHP files with short echo tags; a small `render()` helper keeps the data flow explicit.
- `<?= ?>` prints a value but does not escape it: `htmlspecialchars()` is what turns `&` into `&` and stops cross-site scripting.
- Output buffering captures a rendered piece into a string, which is how layouts and partials compose.
- Twig compiles its own syntax into PHP, escapes every `{{ }}` by default, and keeps logic out of the template through a small set of tags.
- Twig whitespace is literal, so tags and newlines decide the exact output.
- Multibyte strings need `mb_*` functions, because `strlen()` counts bytes and not characters.
- `DateTimeImmutable` is the safe way to work with dates: methods return new objects instead of mutating the one you hold.

8.2 Concepts

Plain PHP templates

Element	Written	Purpose
Short echo	`<?= $title ?>`	print a value
Escaped echo	`<?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?>`	print safely
Condition	`<?php if ($items): ?>`	branch inside the markup

Element	Written	Purpose
Loop	`<?php foreach ($items as $i): ?>`	repeat a block
Include	`<?php include 'partial.php'; ?>`	reuse a fragment

A template receives its variables from the code that renders it and should not fetch data on its own.

The render helper

output

```
render('templates/list.php', ['title' => $t, 'items' => $rows])
         |                                 |
         v                                 v
extract() puts $title and $items in scope, then the
template file is required, so only its output
reaches the browser
```

`EXTR_SKIP` in `extract()` prevents a template variable from overwriting a local one by accident.

Escaping rules

Context	Function
HTML body and attributes	`htmlspecialchars($s, ENT_QUOTES, 'UTF-8')`
URL attribute	`rawurlencode()` before printing
JavaScript value	`json_encode()` inside a script tag
Twig	`{{ value }}` is escaped automatically by the `html` strategy

Escaping happens on output, once, at the last moment; escaping on input would double-escape on the way back out.

Twig syntax

Syntax	Meaning
`{{ value }}`	print, escaped
`{{ value }}` with a filter such as `upper`	apply a filter
`{% if x %} ... {% endif %}`	condition
`{% for item in items %} ... {% endfor %}`	loop
`{% extends 'layout.twig' %}`	inherit a layout
`{% block content %}{% endblock %}`	mark a replaceable area
`{% include 'part.twig' %}`	include a partial
`{# comment #}`	comment, never printed

Because `{{ }}` escapes by default, forgetting the escape is much rarer in Twig than in plain PHP.

Multibyte text

Function	What it counts or cuts
`mb_strlen($s)`	characters, not bytes
`mb_substr($s, 0, 5)`	first five characters
`mb_str_split($s, 5)`	chunks of characters
`mb_strtolower($s)`	locale-aware lower case where the locale applies
`mb_strpos($s, 'ö')`	character position of a needle

Set the internal encoding once, for example in the bootstrap file, so every call agrees on what a character is.

Dates

Need	Call
Now	`new DateTimeImmutable()`
A fixed moment	`new DateTimeImmutable('2026-10-10 09:00')`
A time zone	`new DateTimeImmutable('now', new DateTimeZone('UTC'))`
Add or subtract	`->modify('+3 days')`, returns a new object
Compare	`$a <=> $b` works, and `diff()` gives a `DateInterval`
Render	`->format('D, d M Y H:i')`
Store	`->format('Y-m-d H:i:s')` in the database, or `DATE_ATOM` in JSON

Mutating the object is impossible, so sharing one instance between callers cannot cause a surprise.

8.3 Examples

Example 1: a plain PHP template, escaped.

PHP

```php
<?php

declare(strict_types=1);

function render(string $template, array $vars = []): string
{
    extract($vars, EXTR_SKIP);
    ob_start();
    require $template;

    return ob_get_clean();
}

echo render(__DIR__ . '/templates/list.php', [
    'title' => 'Links & Guides',
    'items' => ['PHP manual', 'Composer docs'],
]);
```

PHP

```php
<h1><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></h1>
<ul>
<?php foreach ($items as $item): ?>
    <li><?= htmlspecialchars($item, ENT_QUOTES, 'UTF-8') ?></li>
<?php endforeach; ?>
</ul>
```

output

```
<h1>Links &amp;amp; Guides</h1>
<ul>
    <li>PHP manual</li>
    <li>Composer docs</li>
</ul>
```

The ampersand in the title arrives as `&`. Without the escape a title containing `<script>` would execute in the browser.

Example 2: a layout assembled with output buffering.

PHP

```php
<?php

declare(strict_types=1);

require __DIR__ . '/helpers.php';

$body = render(__DIR__ . '/templates/list.php', [
    'title' => 'Links & Guides',
    'items' => ['PHP manual'],
]);

echo render(__DIR__ . '/templates/layout.php', ['body' => $body]);
```

PHP

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Links</title>
</head>
<body>
<?= $body ?>
</body>
</html>
```

output

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Links</title>
</head>
<body>
<h1>Links &amp;amp; Guides</h1>
<ul>
    <li>PHP manual</li>
</ul>
</body>
</html>
```

`ob_start()` collects the template output, `ob_get_clean()` returns it as a string and stops buffering, so `$body` can be handed to the layout.

Example 3: the same page in Twig.

PHP

```
<?php

declare(strict_types=1);

require __DIR__ . '/vendor/autoload.php';

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

$twig = new Environment(new FilesystemLoader(__DIR__ . '/templates'));

echo $twig->render('tag.twig', [
    'name' => 'php & friends',
    'tags' => ['pdo', 'composer'],
]);
```

TWIG

```
<p>{{ name|upper }} ({{ tags|length }} tags)</p>
<ul>{% for tag in tags %}<li>{{ tag }}</li>{% endfor %}</ul>
```

output

```
<p>PHP &amp;amp; FRIENDS (2 tags)</p>
<ul><li>pdo</li><li>composer</li></ul>
```

No `htmlspecialchars()` appears, because the `html` autoescape strategy is on by default. Putting the loop on a single line is what keeps the output free of the blank lines that newlines inside a block produce.

Example 4: characters, not bytes.

PHP

```php
<?php

declare(strict_types=1);

$text = 'Héllo, wörld — ☕ time';

echo mb_strlen($text), PHP_EOL;
echo mb_substr($text, 0, 11), PHP_EOL;
echo mb_str_split($text, 5)[1], PHP_EOL;
echo mb_strtolower('STRAßE'), PHP_EOL;
```

output

```
21
Héllo, wörl
, wör
straße
```

`strlen()` would report a much larger number for the same string, because it counts the bytes of the UTF-8 encoding rather than the characters.

Example 5: a deadline with `DateTimeImmutable`.

PHP

```php
<?php

declare(strict_types=1);

$start = new DateTimeImmutable('2026-10-10 09:00', new DateTimeZone('UTC'));
$deadline = $start->modify('+3 days')->setTime(17, 30);

echo $deadline->format('D, d M Y H:i'), PHP_EOL;

$diff = $start->diff($deadline);
printf('%d days, %d hours, %d minutes%s',
    $diff->days, $diff->h, $diff->i, PHP_EOL);
```

output

```
Tue, 13 Oct 2026 17:30
3 days, 8 hours, 30 minutes
```

`modify()` and `setTime()` both return new objects, so `$start` still points at the original moment after both calls.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<?= $row['title'] ?>` with no escape	stored scripts run in the browser	escape with `htmlspecialchars()` or Twig
Escaping on input as well as output	double-escaped text in the browser	escape once, on output
Business logic in a template	logic duplicated across pages	prepare data in the controller
`strlen()` and `substr()` on user text	broken words and cut characters	`mb_strlen()` and `mb_substr()`
Newlines inside Twig block tags	surprising blank lines	keep block tags inline or use trim modifiers
Mutating a shared `DateTime`	one caller's change is another's bug	use `DateTimeImmutable`

- Pass only what a template needs; an array of everything makes the template contract impossible to read.
- Localise formatting at the edge: keep database timestamps in one format and render them per user.

8.5 Practice exercises

1. Which function escapes a value for an HTML attribute, and which flags does it need?
2. What do `ob_start()` and `ob_get_clean()` do together?
3. Why is `<?= $title ?>` not enough?
4. Which two functions replace `strlen()` and `substr()` for multibyte text?
5. Why does `modify()` on a `DateTimeImmutable` return a new object?

Solutions

1. `htmlspecialchars($value, ENT_QUOTES, 'UTF-8')`: the quotes flag escapes both quote styles, and the charset argument makes the encoding explicit.
2. `ob_start()` starts capturing output, and `ob_get_clean()` returns what was captured and ends buffering, so a rendered template becomes a string.
3. The short echo tag prints the raw value; without escaping, markup inside the value becomes live markup.
4. `mb_strlen()` and `mb_substr()` (with `mb_str_split()` for chunking).
5. Because the class is immutable: the original object must stay unchanged so that any other holder still sees the same moment.

8.6 Mini test

1. Which tag prints an escaped value in Twig? A) `{% value %}` B) `{{ value }}` C) `<?= value ?>` D) `echo value`

2. What does `htmlspecialchars('a & b', ENT_QUOTES, 'UTF-8')` return? A) `a & b` B) `a & b` C) `a & b` D) `<b>`
3. Which function captures a rendered template into a string? A) `ob_start()` alone B) `ob_get_contents()` followed by `ob_end_clean()` C) `file_get_contents()` D) `extract()`
4. True or false: Twig escapes `{{ value }}` by default.
5. What does `mb_strlen('éé')` return? A) 4 B) 2 C) 1 D) it depends on the browser
6. Which method adds days without changing the original object? A) `->add()` on a `DateTime` B) `->modify()` on a `DateTimeImmutable` C) `->setDate()` on a `DateTime` D) `->plusDays()` on a `DateInterval`

Answer key and explanations

1. B - double braces print, and the default `html` autoescape strategy converts special characters.
2. B - the ampersand becomes the entity, which is what the browser needs inside text and attributes.
3. B - capturing requires starting the buffer, reading it, and ending it; `ob_start()` alone only starts buffering.
4. True - the `html` strategy applies to every printed expression unless a filter is given explicitly.
5. B - `mb_strlen` counts characters, and the string holds two of them even though each takes two bytes in UTF-8.
6. B - `DateTimeImmutable` methods return a new object, so the original is untouched.

Chapter 9 - Errors and exceptions

9.1 Summary

- An error and an exception are both `Throwable` objects, but errors come from the engine and exceptions from the application.
- `try` wraps a risky block, `catch` receives the exception, and `finally` runs in both cases.
- Several `catch` blocks are tried in order, so the most specific class must come first.
- Custom exceptions extend `Exception` or one of the SPL exceptions, and a named exception documents the failure.
- Rethrowing keeps the original trace while letting another layer react.
- `set_exception_handler()` catches whatever nobody caught, which is the last chance to log and answer politely.
- `set_error_handler()` converts engine warnings and notices into `ErrorException` objects, so they can be handled like any other failure.
- `display_errors` belongs off in production and on in development, while `log_errors` stays on everywhere.

9.2 Concepts

The hierarchy

output

```
                    Throwable (interface)
                   /                     \
              Error                       Exception
             /    \                      /         \
    TypeError   DivisionByZeroError   LogicException  RuntimeException
                                                |
                                        custom domain exceptions
```

| | `Error` | `Exception` | |-|---------|-------------| | Where it comes from | the engine: type faults, arithmetic, undefined calls | application code | | Meaning | a bug in the program | a condition the program must handle | | Example | calling `strlen()` on an array | a payment that was declined |

`Throwable` is the interface both implement, so `catch (Throwable $e)` is the widest useful net.

Handling

Keyword	Behaviour
`try`	marks the block whose exceptions are watched
`catch (Class $e)`	handles one class and its children
`catch (A or B $e)`	handles a union of classes
`finally`	runs after the try or catch, success or failure
`throw`	raises the exception, possibly `$e` again
`e->getMessage()`, `e->getCode()`	what happened, and a machine-readable code
`e->getFile()`, `e->getLine()`	where it was raised
`e->getTraceAsString()`	the call path

A `finally` block runs even when the `try` block returns, and it does not see the value being returned unless it asks for it.

Custom exceptions

Rule	Why
Name the domain failure	`LinkNotFound` reads better than `RuntimeException`
Extend an SPL exception	callers that handle `RuntimeException` keep working
Add a named constructor	`LinkNotFound::withId(42)` builds a well-formed message
Never swallow a caught exception silently	a swallowed bug is a bug that returns

Global handlers

Function	Job
`set_exception_handler(callable)`	handles what nobody caught
`set_error_handler(callable)`	turns warnings and notices into thrown exceptions
`register_shutdown_function(callable)`	runs at the very end, even after a fatal error
`error_log($message)`	writes to the configured error log, wherever it lives
`restore_error_handler()`	puts the previous handler back

Reporting configuration

Directive	Development	Production
`error_reporting`	`E_ALL`	`E_ALL`
`display_errors`	`On`	`Off`
`display_startup_errors`	`On`	`Off`
`log_errors`	`On`	`On`
`error_log`	a path you can read	the system log

`error_reporting` decides what is recorded; `display_errors` decides what the visitor sees. Keeping them separate is what lets you log everything while showing nothing.

9.3 Examples

Example 1: an engine error, caught.

PHP

```php
<?php

declare(strict_types=1);

function divide(int $a, int $b): int
{
    if ($b === 0) {
        throw new DivisionByZeroError('division by zero');
    }

    return intdiv($a, $b);
}

try {
    echo divide(10, 0), PHP_EOL;
} catch (DivisionByZeroError $e) {
    echo 'math error: ', $e->getMessage(), PHP_EOL;
}
```

output

```
math error: division by zero
```

`intdiv(10, 0)` would throw the same class by itself; the explicit check lets you choose the message.

Example 2: a named exception with a factory method.

PHP

```php
<?php

declare(strict_types=1);

class LinkNotFound extends RuntimeException
{
    public static function withId(int $id): self
    {
        return new self(sprintf('link %d not found', $id));
    }
}

function findLink(int $id): array
{
    throw LinkNotFound::withId($id);
}

try {
    findLink(42);
} catch (LinkNotFound $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
} catch (RuntimeException $e) {
    echo 'generic: ', $e->getMessage(), PHP_EOL;
}
```

output

```
LinkNotFound: link 42 not found
```

The specific `catch` comes first, so it wins; the second one would handle any other runtime exception. Putting them in the opposite order makes the second block unreachable.

Example 3: `finally` always runs.

PHP

```php
<?php

declare(strict_types=1);

try {
    echo 'opening', PHP_EOL;
    throw new RuntimeException('boom');
} catch (RuntimeException $e) {
    echo 'caught: ', $e->getMessage(), PHP_EOL;
} finally {
    echo 'always runs', PHP_EOL;
}
```

output

```
opening
caught: boom
always runs
```

A `finally` block is where connections are closed and temporary files deleted, because the code runs on both the success and the failure path.

Example 4: the last-chance handler.

PHP

```php
<?php

declare(strict_types=1);

error_reporting(E_ALL);
ini_set('display_errors', '0');
ini_set('log_errors', '1');

set_exception_handler(static function (Throwable $e): void {
    error_log($e->getMessage());
    http_response_code(500);
    echo 'Something went wrong. Try again later.', PHP_EOL;
});

throw new RuntimeException('database is down');
```

output

```
Something went wrong. Try again later.
```

The message goes to the error log instead of the screen, so an attacker learns nothing while the developer can still read the cause.

Example 5: turn a warning into an exception.

PHP

```php
<?php

declare(strict_types=1);

set_error_handler(static function (int $severity, string $message): bool {
    throw new ErrorException($message, 0, $severity);
});

try {
    $data = json_decode('{bad json', true);
    echo $data === null ? 'decode failed' : 'decoded', PHP_EOL;
} catch (ErrorException $e) {
    echo 'warning became exception: ', $e->getMessage(), PHP_EOL;
} finally {
    restore_error_handler();
}
```

output

```
warning became exception: Syntax error
```

Malformed JSON raises a warning and returns `null`; with the handler installed the same situation becomes a catchable exception, and `restore_error_handler()` puts the default behaviour back afterwards.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
An empty `catch` block	failures disappear and debugging starts from nothing	log, rethrow, or handle deliberately
Catching `Throwable` everywhere	real bugs are hidden behind polite messages	catch the specific class you can handle
No `finally` for cleanup	connections stay open after a failure	close resources in `finally`
`die()` in library code	callers cannot recover or test	throw an exception, let the entry point decide
`display_errors=On` in production	paths, SQL and secrets appear on screen	log errors, show a generic message
Catching and continuing a transaction	the connection is left mid-transaction	roll back in the `catch`

- Add context to a rethrown exception with `new HttpException('message', 0, $e)` so the previous exception stays in the chain.
- A fatal error that kills the process bypasses `catch` entirely, which is why `register_shutdown_function()` is the place to log it.

9.5 Practice exercises

1. Which interface do `Error` and `Exception` both implement?
2. Why must the most specific `catch` block come first?
3. What are the two global handlers used for uncaught exceptions and for engine warnings?
4. Which two directives control what the visitor sees and what is written to the log?
5. Why is `finally` the right place to close a connection?

Solutions

1. `Throwable`.
2. Because `catch` blocks are tested in order and a `catch` for a parent class would also match the child, making the later specific block unreachable.
3. `set_exception_handler()` for uncaught exceptions and `set_error_handler()` for warnings and notices.
4. `display_errors` decides what the visitor sees, and `log_errors` decides what is recorded, together with `error_log` for the destination.
5. Because it runs whether the `try` block succeeded or threw, so the resource is released on every path.

9.6 Mini test

1. Which class is thrown for a scalar type fault? A) `TypeException` B) `TypeError` C) `InvalidArgumentException` D) `DomainException`
2. What does `catch (Throwable $e)` match? A) exceptions only B) errors only C) both errors and exceptions D) everything, including notices
3. True or false: a `finally` block runs after a `return` inside the `try`.
4. Which setting hides error details from visitors in production? A) `error_reporting = E_ALL` B) `display_errors = Off` C) `log_errors = On` D) `error_log = syslog`
5. What is the recommended reaction to a caught exception you cannot handle? A) ignore it B) exit immediately C) log it and rethrow it, or wrap it in a richer exception D) print it to the browser
6. Which function installs a callback for unhandled exceptions? A) `set_error_handler()` B) `set_exception_handler()` C) `register_shutdown_function()` D) `restore_exception_handler()`

Answer key and explanations

1. B - `TypeError` is the engine's `Error` subclass for a value that does not match a declared type.
2. C - `Throwable` is the common interface of the `Error` and `Exception` branches of the hierarchy.
3. True - `finally` executes before control actually leaves the function.
4. B - details go to the log, while the visitor sees a generic response.
5. C - logging preserves the evidence and rethrowing lets a higher layer decide; silencing is the one option that is never right.
6. B - `set_exception_handler()` receives whatever nobody caught, which makes it the last line of defence.

Chapter 10 - Security in practice

10.1 Summary

- The rule behind every item in this chapter is simple: data arriving from outside is unverified until you verify it, and data leaving your application is unencoded until you encode it.
- `password_hash()` and `password_verify()` implement the hashing and the comparison, so the storage format can improve without breaking old hashes.
- `filter_var()` validates that a value has the shape you expect and strips what it should never contain.
- Cross-site scripting is prevented by escaping on output, once, at the point where text becomes markup.
- SQL injection is prevented by prepared statements, never by quoting by hand.

- Cross-site request forgery is prevented by an unpredictable token in the form, checked with `hash_equals()` on the server.
- Sessions must be hardened: `httponly`, `secure` and `SameSite` flags, plus a new session id whenever privileges change.
- Secrets belong in environment variables or in a configuration file outside the web root, never in the repository and never in an error message.

10.2 Concepts

Filtering and validation

Filter	Answers
`FILTER_VALIDATE_EMAIL`	is this a well-formed address
`FILTER_VALIDATE_INT`	is this an integer, possibly in a range
`FILTER_VALIDATE_URL`	is this a well-formed URL
`FILTER_VALIDATE_BOOL`	is this `true`, `on`, `yes`, `1`
`FILTER_VALIDATE_IP`	is this an IPv4 or IPv6 address
`FILTER_DEFAULT`	return the value unchanged

A filter returns the cleaned value, `false` on failure, or `null` when the input is missing, so all three must be distinguished.

Password hashing

Step	Call
Hash on registration	`password_hash($password, PASSWORD_DEFAULT)`
Check on login	`password_verify($password, $hash)`
Upgrade the cost	`password_needs_rehash($hash, PASSWORD_DEFAULT)`

The algorithm behind `PASSWORD_DEFAULT` may change over time, and `password_hash()` generates a random salt for every password, so rainbow tables and shared-hash attacks do not apply. Storing anything yourself, from `md5()` to a hand-rolled HMAC, is strictly worse.

The CSRF round trip

output

```
GET /form                 POST /form
   |                         |
render hidden input       read $_POST['token']
with $_SESSION['token']    compare with hash_equals()
   |                         |
   +--- same browser ------->  equal? continue : reject
```

`hash_equals()` compares in constant time, which stops a timing attack from learning the token one character at a time.

Sessions

Setting	Why
`session_set_cookie_params(['httponly' => true])`	scripts cannot read the cookie
`'secure' => true`	the cookie is sent over HTTPS only
`'samesite' => 'Lax'`	the cookie is not sent on cross-site posts
`session_regenerate_id(true)` after login	the old session id is deleted
`session.use_strict_mode = 1`	uninitialised ids are not accepted

Start the session before any output, because the cookie travels in a header.

Configuration and error reporting

Practice	Reason
Read secrets with `getenv()`	the same code runs in every environment
Keep the configuration file outside the web root	a misconfigured server cannot serve it
Commit a `.env.example` with keys but no values	new developers know what to set
`display_errors = Off` in production	stack traces reveal paths, SQL and secrets
`log_errors = On` everywhere	failures remain visible to developers
`expose_php = Off`	the server stops advertising its PHP version

10.3 Examples

Example 1: hashing, verifying, and upgrading.

PHP

```php
<?php

declare(strict_types=1);

$password = 'correct horse battery staple';
$hash = password_hash($password, PASSWORD_DEFAULT);

var_dump(password_verify($password, $hash));
var_dump(password_verify('wrong password', $hash));

if (password_needs_rehash($hash, PASSWORD_DEFAULT)) {
    $hash = password_hash($password, PASSWORD_DEFAULT);
}

echo 'stored length: ', strlen($hash), PHP_EOL;
```

output

```
bool(true)
bool(false)
stored length: 60
```

The hash differs on every run because the salt is random; the length is stable for the default algorithm, and no part of the password appears in it.

Example 2: validate before you use a value.

PHP

```php
<?php

declare(strict_types=1);

$values = ['ada@example.com', 'not-an-email', '42', '7 puppies'];

foreach ($values as $value) {
    if (filter_var($value, FILTER_VALIDATE_EMAIL) !== false) {
        echo $value, ' is an address', PHP_EOL;
    } elseif (filter_var($value, FILTER_VALIDATE_INT) !== false) {
        echo $value, ' is an integer', PHP_EOL;
    } else {
        echo $value, ' is neither', PHP_EOL;
    }
}
```

output

```
ada@example.com is an address
not-an-email is neither
42 is an integer
7 puppies is neither
```

Note the strict comparisons: `filter_var()` returns `false` for a failure and can legitimately return `0`, which a loose test would confuse with `false`.

Example 3: a CSRF token, issued and checked.

PHP

```php
<?php

declare(strict_types=1);

session_start();

if (empty($_SESSION['token'])) {
    $_SESSION['token'] = bin2hex(random_bytes(32));
}

$posted = $_POST['token'] ?? '';
var_dump(hash_equals($_SESSION['token'], $posted));

$posted = $_SESSION['token'];
var_dump(hash_equals($_SESSION['token'], $posted));
```

The hidden field that carries the token:

PHP

```php
<input type="hidden" name="token"
       value="<?= htmlspecialchars($token, ENT_QUOTES, 'UTF-8') ?>">
```

output

```
bool(false)
bool(true)
```

An attacker's page can copy the form but cannot read the session, so it can only send a wrong token.

Example 4: harden the session cookie.

PHP

```php
<?php

declare(strict_types=1);

session_set_cookie_params([
    'lifetime' => 0,
    'httponly' => true,
    'secure' => true,
    'samesite' => 'Lax',
]);

session_start();
session_regenerate_id(true);

$_SESSION['user'] = 'ada';
echo 'logged in as ', $_SESSION['user'], PHP_EOL;
```

output

```
logged in as ada
```

Calling `session_regenerate_id(true)` issues a fresh identifier and deletes the old one, which is what stops an attacker who fixed the victim's session id before the login from inheriting it afterwards.

Example 5: read configuration from the environment.

PHP

```php
<?php

declare(strict_types=1);

$key = getenv('APP_SECRET');

if ($key === false || $key === '') {
    fwrite(STDERR, 'APP_SECRET is not set', PHP_EOL);
    exit(1);
}

$dsn = getenv('DB_DSN') ?: 'sqlite:' . __DIR__ . '/var/app.db';

echo 'configuration loaded', PHP_EOL;
```

INI

```
display_errors = Off
display_startup_errors = Off
log_errors = On
error_log = /var/log/php-errors.log
expose_php = Off
```

Fail fast and mention the variable name, never its value, in the message.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`md5()` or `sha1()` for passwords	cracked in minutes on commodity hardware	`password_hash()` and `password_verify()`
Comparing tokens with `==` or `===` on secrets	timing leaks	`hash_equals()`
Trusting a hidden form field	it is only a convenience for the user	re-validate everything on the server
Printing the database DSN in an error page	credentials on a public screen	log details, show a generic message
Keeping the same session id after login	session fixation	`session_regenerate_id(true)`
Committing `.env` next to the code	every clone and fork carries the secret	commit an example file, inject real values
Escaping only some fields	the missed field is the exploited one	escape centrally, or use a template engine that escapes

- Assume every list of "common mistakes" is incomplete: treat any input as hostile and any output as a place where markup can be injected.
- Keep dependencies current with `composer audit`; most real-world breaches start with a known vulnerability in an unpatched package.

10.5 Practice exercises

1. Which two functions hash a password and check it again?
2. Why is a random salt stored inside the hash rather than kept separately?
3. What does `hash_equals()` protect against?
4. What must happen to the session immediately after a successful login?
5. Where should a database password live in a project that is in version control?

Solutions

1. `password_hash($password, PASSWORD_DEFAULT)` and `password_verify($password, $hash)`.
2. Because the salt has to be different for every password and it must be available when checking; storing it with the hash keeps the pair together and the code simple.

3. A timing attack: the comparison takes the same time whether the mismatch is at the first character or the last.
4. `session_regenerate_id(true)`, so a session id known before the login cannot be used afterwards.
5. In an environment variable or in a configuration file outside the web root; the repository holds only an example file with the keys.

10.6 Mini test

1. Which function verifies a password against a stored hash? A) `password_check()` B) `password_verify()` C) `password_compare()` D) `hash_equals()`
2. What does `PASSWORD_DEFAULT` refer to? A) a fixed algorithm that never changes B) the current recommended algorithm C) the fastest available algorithm D) a plain-text mode
3. Which function should compare a CSRF token? A) `strcmp()` B) `hash_equals()` C) `strcasecmp()` D) `==`
4. True or false: a hidden form field is a valid way to trust a value.
5. Which combination is correct for a production server? A) `display_errors=On`, `log_errors=Off` B) `display_errors=Off`, `log_errors=On` C) both off D) both on
6. What prevents SQL injection in a query with user input? A) quoting the value by hand B) a prepared statement with placeholders C) an escape function from a tutorial D) a shorter query

Answer key and explanations

1. B - it recomputes the hash with the salt stored in the given hash and compares them safely.
2. B - the constant points at the algorithm the running PHP version recommends, so a future version can choose a stronger one.
3. B - a constant-time comparison that cannot be shortened by guessing one byte at a time.
4. False - anything in the request can be edited by the client, so a hidden field is a convenience, never an assertion.
5. B - details are logged while the visitor sees only a generic message.
6. B - values travel separately from the SQL text, so no input can change the statement's structure.

Chapter 11 - Testing with PHPUnit

11.1 Summary

- A test is code that runs another piece of code and checks the result, so a regression is caught by a machine instead of by a user.

- PHPUnit is the standard test framework; it reads `phpunit.xml`, discovers classes ending in `Test.php`, and reports pass, fail and error separately.
- `assertSame()` compares type and value, while `assertEquals()` allows type juggling, which is why the strict version finds more bugs.
- `expectException()` and `expectExceptionMessage()` test that a failure path raises the right exception.
- A data provider feeds the same test with many inputs, which keeps the test body written once.
- Test doubles replace collaborators: a stub answers questions and a mock also records expectations about calls.
- Test-driven development writes the test first, watches it fail, then writes the smallest code that makes it pass.
- Behaviour-driven development describes the same idea in sentences for non-developers and is complementary, not competing.

11.2 Concepts

Kinds of tests

Kind	Scope	Speed
Unit	one class, collaborators replaced	milliseconds
Integration	several classes plus a real database or filesystem	slower
End-to-end	the whole application through its entry point	slowest

A healthy suite has many unit tests, fewer integration tests and a handful of end-to-end tests.

Layout

output

```
composer.json        phpunit.xml          tests/
phpunit in          the bootstrap and    SluggerTest.php
require-dev         the test directory   LinkCounterTest.php
                           |
                           v
                    vendor/bin/phpunit
```

The `bootstrap` attribute loads `vendor/autoload.php`, so both `Acme\` and `Acme\Tests\` resolve without any extra code.

Assertions you will use first

Assertion	Checks
`assertSame($expected, $actual)`	same type and value
`assertEquals($expected, $actual)`	same value, loose comparison
`assertTrue()`, `assertFalse()`	a boolean expression

Assertion	Checks
`assertNull()`, `assertNotNull()`	a value is or is not `null`
`assertCount($n, $array)`	the array has `$n` elements
`assertInstanceOf(Class::class, $obj)`	the type of an object
`assertStringContainsString('a', $s)`	a substring occurs
`assertSame([...], $actual)`	whole arrays compared strictly
`expectException(Class::class)`	the code throws this class
`expectExceptionMessage('text')`	and the message contains this

Doubles

Double	Behaviour
`createStub(Class::class)`	every method returns a type-appropriate empty value unless configured
`createMock(Class::class)`	like a stub, and it also records and verifies calls
Implemented fake class	a small class with the behaviour you want, no framework

Stubs answer the question "what does my code do with this collaborator"; mocks answer the stricter question "does my code call the collaborator correctly".

11.3 Examples

Example 1: the test configuration.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         bootstrap="vendor/autoload.php">
    <testsuites>
        <testsuite name="unit">
            <directory suffix="Test.php">tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```

Running `vendor/bin/phpunit` from the project root now discovers every test file in `tests/` without passing a path.

Example 2: the class under test and its first test.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme;

final class Slugger
{
    public function slug(string $title): string
    {
        $slug = strtolower(trim($title));
        $slug = preg_replace('/[^a-z0-9]+/', '-', $slug);

        return trim($slug, '-') ?: 'link';
    }
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Tests;

use Acme\Slugger;
use PHPUnit\Framework\TestCase;

final class SluggerTest extends TestCase
{
    public function testSlugifiesATitle(): void
    {
        self::assertSame('hello-php', (new Slugger())->slug('Hello, PHP!'));
    }

    public function testEmptyResultFallsBack(): void
    {
        self::assertSame('link', (new Slugger())->slug('***'));
    }
}
```

Test methods are `public`, start with `test`, take no arguments, and return `void`. Two tests mean two independent checks, so one failing does not hide the other.

Example 3: run the suite.

shell

```
vendor/bin/phpunit
```

output

```
PHPUnit 10.5.38 by Sebastian Bergmann and contributors.

.                                                                   1 / 1 (100%)

Time: 00:00.011, Memory: 8.00 MB

OK (1 test, 1 assertion)
```

A failure prints the expected and the actual value side by side, which is usually enough to see the bug without opening a debugger.

Example 4: the same test, many inputs.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Tests;

use Acme\Slugger;
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class SluggerDataTest extends TestCase
{
    #[DataProvider('titles')]
    public function testSlugIsAlwaysLowercase(string $input, string $expected): void
    {
        self::assertSame($expected, (new Slugger())->slug($input));
    }

    public static function titles(): array
    {
        return [
            'spaces' => ['My First Link', 'my-first-link'],
            'symbols' => ['PHP & SQL', 'php-sql'],
            'trimmed' => ['  padded  ', 'padded'],
        ];
    }
}
```

Each array entry becomes one test run, named by its key, so a failure names exactly the input that broke.

Example 5: replace a collaborator with a stub.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Tests;

use Acme\Entity\Category;
use Acme\Entity\Link;
use Acme\LinkCounter;
use Acme\Repository\LinkRepository;
use PHPUnit\Framework\TestCase;

final class LinkCounterTest extends TestCase
{
    public function testCountsEveryLink(): void
    {
        $repository = $this->createStub(LinkRepository::class);
        $repository->method('all')->willReturn([
            new Link('PHP manual', 'https://www.php.net', Category::Docs),
            new Link('Composer', 'https://getcomposer.org', Category::Docs),
        ]);

        $counter = new LinkCounter($repository);

        self::assertSame(2, $counter->count());
    }
}
```

No database, no network, no file: the repository is a stub, so the test runs in a millisecond and always passes or fails for the same reason.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Testing a method with no assertion	the test passes whatever happens	assert a concrete value or exception
`assertEquals()` for exact values	`0` and `'0'` pass, which hides a bug	`assertSame()` unless you mean otherwise
Tests that depend on each other	one failure cascades into five	each test sets up its own data
Hitting the real database in a unit test	slow, flaky, order-dependent	stub the repository, use a real one in integration tests
Asserting on private internals	refactoring breaks the tests	assert on observable behaviour
Skipping the failing-first step	the test may not test anything	watch it fail before writing the code

- Name a test after the behaviour it proves: `testEmptyTitleFallsBackToLink()` reads like a sentence six months later.
- Run the suite on every commit; a suite nobody runs is decoration.

11.5 Practice exercises

1. Which file tells PHPUnit where the autoloader is?
2. What is the difference between `assertSame` and `assertEquals`?
3. How do you check that a method throws a specific exception?
4. When should a test use a stub instead of a real database?
5. What are the three steps of test-driven development?

Solutions

1. `phpunit.xml`, through its `bootstrap` attribute.
2. `assertSame` compares type and value; `assertEquals` accepts values that are equal after type juggling.
3. Call `$this->expectException(Class::class)` before the line that throws, optionally with `$this->expectExceptionMessage('...')`.
4. When the test is a unit test for the class that uses the repository: the repository is a stub, and a real database is left to the integration suite.
5. Write a failing test, make it pass with the simplest change, then refactor while keeping the tests green.

11.6 Mini test

1. Which class does a PHPUnit test extend? A) `PHPUnit\TestCase` B) `PHPUnit\Framework\TestCase` C) `PHPUnit\Assert` D) `TestCase`
2. Which assertion is strict about types? A) `assertEquals()` B) `assertSame()` C) `assertEquivalent()` D) `assertSimilar()`
3. What does a data provider return? A) a single array of values B) an array of rows, each one a set of arguments C) a generator of test names D) a callback
4. True or false: `createStub()` also verifies that a method was called.
5. Which is a sign of a good unit test? A) it needs a live database B) it runs in milliseconds and depends on nothing external C) it asserts on private properties D) it calls `sleep()` between steps
6. What is the recommended order in test-driven development? A) code, then test, then delete the test B) failing test, passing code, refactor C) refactor, then code, then test D) test, code, then never refactor

Answer key and explanations

1. B - the fully qualified base class is `PHPUnit\Framework\TestCase`.
2. B - `assertSame` fails when the types differ, so it catches `0` versus `'0'` mistakes.
3. B - each row of the array is one call of the test method, and a string key names that run in the report.
4. False - a stub only answers calls; verification of the calls is what a mock adds.

5. B - no external service, no timing dependency, and a single reason to fail.
6. B - the failing test proves the test works, the minimal change proves the code works, and refactoring keeps the design clean.

Chapter 12 - Deployment, caching and the ecosystem

12.1 Summary

- A deployment installs the locked dependencies, writes the configuration the environment provides, applies migrations, and warms the caches.
- `composer install --no-dev --optimize-autoloader` is the standard install command on a server, because development packages and the debugger belong to your laptop.
- PHP itself needs no build step, which is why "building" a PHP application usually means validating, testing and warming caches.
- OPcache keeps compiled bytecode in shared memory, so a request does not recompile the same files on every hit.
- Object caching remembers computed results, and HTTP caching tells browsers and proxies what they may reuse.
- Docker packages the runtime with the application; Vagrant did the same with a virtual machine and is still common on older teams.
- PHPDoc comments describe types and behaviour for readers and for tools such as static analysers and IDEs.
- The ecosystem is coordinated through PHP-FIG recommendations, so packages from different authors can be swapped without rewriting your code.

12.2 Concepts

Deployment steps

Step	Command or action
Fetch the code	`git pull --ff-only` or upload a release
Install dependencies	`composer install --no-dev --optimize-autoloader`
Provide configuration	environment variables or a file outside the web root
Apply migrations	a migration tool, once, not on every request
Warm caches	rebuild templates, opcache reset, precompute values
Verify	`php -l`, run the test suite, then switch traffic

Point releases and quick rollbacks are much easier when each deployment writes a new directory and switches a symbolic link, but the same steps apply either way.

The caching layers

Layer	What it avoids	Tool
Opcode	recompiling PHP files	OPcache
Object	recomputing a query or an API call	APCu, Redis, Memcached
HTTP	re-downloading unchanged responses	`Cache-Control`, `ETag`
Page	running PHP at all for a cached page	a reverse proxy in front of the app

Cache invalidation is the hard part: every entry needs a rule for when it becomes wrong, usually a time to live or an explicit purge on write.

OPcache in one table

Directive	Effect
`opcache.enable=1`	turn the cache on
`opcache.memory_consumption=128`	megabytes of shared memory for compiled code
`opcache.max_accelerated_files`	how many files may be cached
`opcache.validate_timestamps=0`	never check file mtimes: fastest, needs a reset on deploy
`opcache.revalidate_freq=0`	check timestamps on every request, for development

With `validate_timestamps=0`, a deployment must reset the cache with `opcache_reset()` or by restarting PHP-FPM, or the new code is never loaded.

Virtualization and containers

Approach	What it gives you
Docker	an image with PHP, extensions, Composer and the app, run the same everywhere
Vagrant	a virtual machine described in a file, handy for old desktop stacks
Platform as a service	a PHP buildpack and a database, no server administration

A container image pins the PHP version and the extensions, so "works on my machine" becomes a property of the build instead of a hope.

PHPDoc

Tag	Use
`@param type $name`	describes a parameter
`@return type`	describes the return value
`@var type`	describes a property
`@throws Exception`	a checked failure path

Tag	Use
`@deprecated`	a feature on its way out

Modern PHP declares types in the signature and uses the docblock for the parts signatures cannot express, such as `list<Link>` or a description of the rule being enforced.

The ecosystem

Recommendation	Subject
PSR-1, PSR-12	coding style
PSR-3	logging interface
PSR-4	autoloading
PSR-7	HTTP message interfaces
PSR-11	container interface
PSR-15	request handlers and middleware
PSR-18	HTTP client interface

Full frameworks, component libraries and single-purpose packages all publish their signatures on Packagist; this volume keeps to the standards so that any of them can be adopted later.

12.3 Examples

Example 1: a minimal container for a PHP-FPM application.

Dockerfile

```
FROM php:8.3-fpm-alpine

WORKDIR /var/www/html
COPY . .

RUN docker-php-ext-install pdo pdo_sqlite

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
RUN composer install --no-dev --optimize-autoloader --no-interaction

RUN chown -R www-data:www-data var
```

The image pins the PHP version and the extensions, and dependencies are installed inside the build so the running container only holds code.

Example 2: the commands a deployment runs.

shell

```
set -e
git pull --ff-only
composer install --no-dev --optimize-autoloader
php -l public/index.php
vendor/bin/phpunit
php -r 'opcache_reset();'
```

The order matters: dependencies before migration, tests before the cache reset, and the opcache reset last so the new bytecode is the one that is served.

Example 3: production OPcache settings.

INI

```ini
opcache.enable=1
opcache.memory_consumption=128
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
opcache.revalidate_freq=0
```

With timestamps disabled nothing is checked on disk, so the deployment script must reset the cache, as in Example 2.

Example 4: a documented class.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Feed;

/**
 * Lists the newest links, newest first.
 */
final class LinkFeed
{
    /**
     * @param  list<string>  $tags  only links with these tags
     * @return list<array{id: int, title: string, url: string}>
     */
    public function latest(array $tags = [], int $limit = 20): array
    {
        return [];
    }
}
```

The `@param` line describes array shapes the signature cannot express, which is what makes a static analyser and an editor useful at the same time.

Example 5: what never belongs in the repository.

Contents of `.gitignore`:

output

```
/vendor/
/var/
.env
*.log
```

Contents of `.env.example`:

output

```
APP_SECRET=
DB_DSN=sqlite:/var/www/html/var/app.db
MAILER_DSN=smtp://localhost:1025
```

The example file carries the keys with empty values, so a new developer knows which variables to set, and the real file is never committed.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`composer update` on the server	production changes without review	`composer install` from the lock file
Installing development packages in production	debuggers and test tools in the live tree	`composer install --no-dev`
OPcache with timestamps checked constantly	slower requests and stale code both possible	`validate_timestamps=0` plus a reset on deploy
No database migration plan	schema changes applied by hand, in the wrong order	one ordered migration file per change
Caching with no invalidation rule	users see wrong data until the TTL passes	define the TTL or purge on write
Secrets committed in a configuration file	every clone and fork carries the secret	environment variables and an example file
No rollback plan	a bad release stays live until someone fixes it	keep the previous release, switch a link back

- Log the deployment steps to a place you can read during an incident.
- Measure one request before and after adding a cache; if the gain is not visible, remove the cache and its invalidation code.

12.5 Practice exercises

1. Which Composer command installs a project on a production server?
2. What does `opcache.validate_timestamps=0` change, and what must the deployment then do?
3. Name the four caching layers described in this chapter.
4. Why is `vendor/` ignored while `composer.lock` is committed?
5. What does a PSR-7 message represent?

Solutions

1. `composer install --no-dev --optimize-autoloader`, which installs exactly the locked versions without development packages.
2. OPcache stops comparing file timestamps, so requests stop paying for the check; the deployment must reset the cache, for example with `opcache_reset()` or by restarting

PHP-FPM.

3. Opcode caching, object caching, HTTP caching and page caching.
4. `vendor/` is a generated directory that would produce enormous diffs, while the lock file records the exact versions that were tested and installed.
5. An HTTP request or response as objects, so middleware and clients can be written without depending on PHP superglobals.

12.6 Mini test

1. Which flag excludes development packages during install? A) `--no-plugins` B) `--no-dev` C) `--no-scripts` D) `--prefer-dist`
2. What does OPcache cache? A) database results B) compiled PHP bytecode C) HTTP responses D) session data
3. True or false: a PHP deployment normally includes a compilation step like the one a compiled language needs.
4. Which file lists the keys of required environment variables without the values? A) `.env` B) `.env.example` C) `php.ini` D) `composer.lock`
5. What is a migration? A) a cache key B) an ordered file describing one schema change C) a Composer script D) a PSR describing HTTP messages
6. Which PSR covers logging? A) PSR-3 B) PSR-4 C) PSR-7 D) PSR-11

Answer key and explanations

1. B - development dependencies are skipped, which also means the debugger and test framework are absent from production.
2. B - the compiled form of a file is kept in shared memory, so the engine does not recompile it for every request.
3. False - PHP runs from source; the deployment installs dependencies and warms caches, and the first request compiles a file only once thanks to OPcache.
4. B - an example file with the keys and empty values documents the configuration without leaking secrets.
5. B - migrations apply schema changes in order, in every environment the same way.
6. A - PSR-3 defines the logging interface that every modern logger implements.

Chapter 13 - Capstone: a links manager

13.1 Summary

- The capstone is a small links manager: a page that lists saved links, a form that adds one, and a repository that stores them in SQLite.
- Every file uses a practice from an earlier chapter: PSR-4 namespaces, typed and readonly code, prepared statements, escaping, a CSRF token and a repository interface.

- The application has one entry point, `public/index.php`, so the web root contains nothing but the front controller.
- A route table of regular expressions is enough for routing at this size, which keeps the project free of framework decisions.
- The front controller wires the object graph once and dispatches to a controller, which is the smallest version of dependency injection.
- The template is a plain PHP file that escapes every value it prints.
- The session holds the CSRF token, and the controller checks it before any write.
- The same code runs under the built-in server for development and under PHP-FPM in production.

13.2 Concepts

The request path

output

```
POST /links
   |
public/index.php        session, PDO, repository, controller, router
   |
Acme\Http\Router        match method + pattern, call the controller
   |
Acme\Http\LinkController validate input, check the token, save
   |
Acme\Repository\        INSERT with placeholders
PdoLinkRepository
   |
templates/links.php     escaped HTML, back to the browser
```

The file layout

output

```
 links/
|-- public/
|   `-- index.php            front controller: wiring and dispatch
|-- src/
|   |-- Entity/
|   |   |-- Link.php         a readonly value object
|   |   `-- Category.php     a backed enum
|   |-- Http/
|   |   |-- Router.php       method + pattern dispatch
|   |   `-- LinkController.php  validation, token check, render
|   |-- Repository/
|   |   |-- LinkRepository.php    the interface
|   |   `-- PdoLinkRepository.php the only SQL in the app
|   |-- Security/
|   |   `-- CsrfToken.php    per-session token, constant-time compare
|   `-- functions.php        the render() helper
|-- templates/
|   `-- links.php            one escaped HTML page
|-- tests/
|   `-- CsrfTokenTest.php    a unit test with no database
|-- var/
|   |-- links.db             the SQLite file, created on demand
|   `-- schema.sql           the table, applied when the app starts
|-- composer.json            autoload, scripts, requirements
`-- phpunit.xml              test configuration
```

What comes from where

Chapter	Practice used here
1	the built-in server, `php -l`
2	PSR-4 autoloading under the `Acme\` prefix
3	readonly properties, a backed enum, named arguments
4	namespaces and `use` imports
5	`composer.json` with `autoload` and `scripts`
6	a repository interface with a PDO implementation
7	prepared statements and an exception-mode connection
8	a plain PHP template that escapes every value
9	errors surface as exceptions, caught at the entry point
10	hashed passwords elsewhere, token checks, config outside the web root
11	a unit test that stubs nothing and still needs no database
12	SQLite for the data, OPcache unrelated to correctness

The route table

Method	Pattern	Handler
GET	`#^/links/?$#`	`LinkController::index`

Method	Pattern	Handler
`POST`	`#^/links/?$#`	`LinkController::index`
anything else	no match	a `404` response from the router

The router compares the method first, then tries the pattern, and passes the captured groups to the handler as arguments. With one controller and two routes the table is honest about what the application does.

13.3 Examples

Example 1: the schema, in a file a migration tool can apply.

SQL

```sql
CREATE TABLE IF NOT EXISTS links (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    url TEXT NOT NULL,
    category TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Example 2: the entity and its enum.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Entity;

enum Category: string
{
    case Docs = 'docs';
    case Video = 'video';
    case Blog = 'blog';
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Entity;

final class Link
{
    public function __construct(
        public readonly string $title,
        public readonly string $url,
        public readonly Category $category,
        public readonly ?int $id = null,
    ) {
    }

    public function withId(int $id): self
    {
        return new self($this->title, $this->url, $this->category, $id);
    }
}
```

The object is immutable and reads like the row it came from.

Example 3: the repository interface and its PDO implementation.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Repository;

use Acme\Entity\Link;

interface LinkRepository
{
    /** @return list<Link> */
    public function all(): array;

    public function save(Link $link): void;
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Repository;

use Acme\Entity\Category;
use Acme\Entity\Link;
use PDO;

final class PdoLinkRepository implements LinkRepository
{
    public function __construct(private PDO $connection)
    {
    }

    public function all(): array
    {
        $rows = $this->connection
            ->query('SELECT id, title, url, category FROM links ORDER BY id DESC')
            ->fetchAll(PDO::FETCH_ASSOC);

        return array_map(
            static fn (array $row): Link => new Link(
                $row['title'],
                $row['url'],
                Category::from($row['category']),
                (int) $row['id'],
            ),
            $rows,
        );
    }

    public function save(Link $link): void
    {
        $statement = $this->connection->prepare(
            'INSERT INTO links (title, url, category) VALUES (?, ?, ?)'
        );

        $statement->execute([
            $link->title,
            $link->url,
            $link->category->value,
        ]);
    }
}
```

This is the only class in the application that contains SQL, and no value is ever interpolated into a query string.

Example 4: the CSRF token helper.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Security;

final class CsrfToken
{
    public function __construct(private string $key = 'csrf_token')
    {
    }

    public function token(): string
    {
        if (empty($_SESSION[$this->key])) {
            $_SESSION[$this->key] = bin2hex(random_bytes(32));
        }

        return $_SESSION[$this->key];
    }

    public function isValid(): bool
    {
        $posted = $_POST[$this->key] ?? '';

        return is_string($posted)
            && isset($_SESSION[$this->key])
            && hash_equals($_SESSION[$this->key], $posted);
    }
}
```

`random_bytes()` comes from the same extension that provides `password_hash()`, so the token is unpredictable, and `hash_equals()` compares without leaking timing information.

Example 5: the router.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Http;

final class Router
{
    /** @var list<array{string, string, callable}> */
    private array $routes = [];

    public function add(string $method, string $pattern, callable $handler): void
    {
        $this->routes[] = [$method, $pattern, $handler];
    }

    public function dispatch(string $method, string $path): void
    {
        foreach ($this->routes as [$routeMethod, $pattern, $handler]) {
            if ($routeMethod !== $method) {
                continue;
            }

            if (preg_match($pattern, $path, $matches) === 1) {
                $handler(...array_slice($matches, 1));
                return;
            }
        }

        http_response_code(404);
        echo 'Not found';
    }
}
```

Route handlers are first-class callables, so `$controller->index(...)` is passed as a value instead of a method name looked up later.

Example 6: the controller and the render helper.

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Http;

use Acme\Entity\Category;
use Acme\Entity\Link;
use Acme\Repository\LinkRepository;
use Acme\Security\CsrfToken;

final class LinkController
{
    public function __construct(
        private LinkRepository $links,
        private CsrfToken $csrf,
    ) {
    }

    public function index(): void
    {
        $title = trim((string) ($_POST['title'] ?? ''));
        $url = trim((string) ($_POST['url'] ?? ''));

        if ($title !== ''
            && filter_var($url, FILTER_VALIDATE_URL) !== false
            && $this->csrf->isValid()
        ) {
            $this->links->save(new Link($title, $url, Category::Docs));
        }

        render(__DIR__ . '/../../templates/links.php', [
            'links' => $this->links->all(),
            'token' => $this->csrf->token(),
        ]);
    }
}
```

PHP

```php
<?php

declare(strict_types=1);

function render(string $template, array $vars = []): void
{
    extract($vars, EXTR_SKIP);
    require $template;
}
```

The controller receives both collaborators through its constructor, so the test in Example 9 can replace the repository without touching PDO.

Example 7: the template.

PHP

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Links</title>
</head>
<body>
    <h1>Links</h1>

    <ul><?php foreach ($links as $link): ?>
        <li><?= htmlspecialchars($link->title, ENT_QUOTES, 'UTF-8') ?>
            <small><?= $link->category->value ?></small></li>
    <?php endforeach; ?></ul>

    <form method="post" action="/links">
        <input type="hidden" name="csrf_token"
               value="<?= htmlspecialchars($token, ENT_QUOTES, 'UTF-8') ?>">
        <label>Title <input name="title" required></label>
        <label>URL <input name="url" type="url" required></label>
        <button type="submit">Save link</button>
    </form>
</body>
</html>
```

Every printed value goes through `htmlspecialchars()`, and the whitespace around the `foreach` is arranged so the rendered list has no blank lines.

Example 8: the front controller.

PHP

```
<?php

declare(strict_types=1);

use Acme\Http\LinkController;
use Acme\Http\Router;
use Acme\Repository\PdoLinkRepository;
use Acme\Security\CsrfToken;

require dirname(__DIR__) . '/vendor/autoload.php';

session_start();

$connection = new PDO('sqlite:' . dirname(__DIR__) . '/var/links.db');
$connection->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$connection->exec(file_get_contents(
    dirname(__DIR__) . '/var/schema.sql'
));

$controller = new LinkController(
    new PdoLinkRepository($connection),
    new CsrfToken(),
);

$router = new Router();
$router->add('GET', '#^/links/?$#', $controller->index(...));
$router->add('POST', '#^/links/?$#', $controller->index(...));

$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$router->dispatch($_SERVER['REQUEST_METHOD'], $path);
```

The schema is applied on every start because the statement is idempotent; a real project uses a migration tool that records which files it already applied.

Example 9: the manifest and a unit test.

JSON

```json
{
    "name": "acme/links",
    "description": "A small links manager",
    "type": "project",
    "license": "MIT",
    "require": {
        "php": ">=8.3",
        "ext-pdo": "*",
        "ext-pdo_sqlite": "*",
        "ext-mbstring": "*"
    },
    "autoload": {
        "psr-4": {
            "Acme\\": "src/"
        },
        "files": [
            "src/functions.php"
        ]
    },
    "scripts": {
        "serve": "php -S localhost:8000 -t public",
        "lint": "phpcs --standard=PSR12 src tests",
        "test": "phpunit"
    }
}
```

PHP

```php
<?php

declare(strict_types=1);

namespace Acme\Tests;

use Acme\Security\CsrfToken;
use PHPUnit\Framework\TestCase;

final class CsrfTokenTest extends TestCase
{
    public function testTokenIsStableForASession(): void
    {
        $_SESSION = [];
        $csrf = new CsrfToken();

        self::assertSame($csrf->token(), $csrf->token());
    }

    public function testWrongTokenIsRejected(): void
    {
        $_SESSION = [];
        $csrf = new CsrfToken();
        $_POST = ['csrf_token' => 'not the token'];

        self::assertFalse($csrf->isValid());
    }
}
```

The test touches only the superglobals it controls, so it runs in isolation and in any order.

Example 10: run the application.

shell

```
composer install
composer serve
```

output

```
PHP 8.3.12 Development Server (http://localhost:8000) started
```

Add two links through the form, then request the page:

shell

```
curl -s http://localhost:8000/links
```

output

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Links</title>
</head>
<body>
    <h1>Links</h1>

    <ul>
        <li>PHP manual
            <small>docs</small></li>
        <li>Composer
            <small>docs</small></li>
    </ul>

    <form method="post" action="/links">
        <input type="hidden" name="csrf_token"
               value="9f2c1d4e7ab0c5f30d1e2a4b6c8d0f2a4b6c8d0f2a4b6c8d0f2a4b6c8d0f2a">
        <label>Title <input name="title" required></label>
        <label>URL <input name="url" type="url" required></label>
        <button type="submit">Save link</button>
    </form>
</body>
</html>
```

The two entries appear newest first, because the query orders by descending id. The token value is random for every session, so it differs on each run. A POST with a missing or wrong token returns the same page without adding anything, because the controller saves only when the token check passes.

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Business logic in the template	the same validation appears in two files	prepare data in the controller

Mistake	Why it hurts	Best practice
SQL spread over controllers	no single place to fix a query	one repository interface with one implementation
Building routes by hand with `if ($_GET['page'] == ...)`	every new page edits the same fragile block	a route table with method and pattern
No CSRF check on the write route	any page can post to the form	check the token before any save
Reading `var/links.db` from outside `public/` but keeping files there too	the web server can serve the database file	keep writable data outside the web root
Skipping the unit test because there is no database	the token logic is the easiest part to break	test the classes that do not need one

- Run `composer lint`, `composer test` and `php -l public/index.php` before every commit; the whole suite runs in under a second.
- When the application grows, keep the boundaries that exist here: a controller per resource, a repository per aggregate, and templates that only print.

13.5 Practice exercises

1. Which file is the only entry point, and why is it the only file in the web root?
2. Where does the CSRF token live, and what happens when the posted token does not match?
3. Add a `DELETE` route that removes the link with a given id. Which classes change?
4. Why does `LinkController` accept an interface rather than a PDO object?
5. What would you change to move this application from SQLite to MySQL?

Solutions

1. `public/index.php`. It is the only web-visible file, so no configuration, template or database file can be requested directly by a browser.
2. In the session, held by `Acme\Security\CsrfToken`. When the posted value does not match, `isValid()` returns `false` and the controller does not save, because the condition requires the token check to pass.
3. A method on the controller and a route line in the front controller; the router already captures pattern groups and passes them to the handler, so a `#^/links/(\d+)/delete$#` pattern gives the controller the id.
4. So the controller depends on the operations it needs, not on PDO, which keeps it testable and lets the storage be replaced without touching it.
5. Only the DSN in the front controller and the driver, since the SQL is parameterised and stays inside the repository; a new `composer.json` requirement for the matching `ext-pdo_mysql` would also be needed.

13.6 Mini test

1. Which directory should the web server use as its document root? A) the project root B) `public/` C) `src/` D) `var/`
2. What is the front controller's main job? A) escaping output B) wiring the objects and dispatching the request C) writing SQL D) hashing passwords
3. Which class is allowed to contain SQL? A) the controller B) the template C) the repository implementation D) the router
4. True or false: the template should call `$this->links->all()` itself.
5. What does the router pass to a handler when a pattern captures a group? A) nothing B) the captured values as arguments C) the whole query string D) the request object
6. Why is the SQLite database kept outside `public/`? A) SQLite cannot open files in the web root B) a misconfigured server could otherwise serve the file itself C) it makes queries faster D) Composer requires it

Answer key and explanations

1. B - `public/` holds only the front controller, so nothing else is reachable over HTTP.
2. B - it builds the connection, repository, controller and router once and then hands control to the router.
3. C - one implementation class, behind an interface, keeps the SQL in a single place.
4. False - a template receives prepared variables; fetching data belongs to the controller or a service.
5. B - captured groups are passed positionally to the handler, which is how an id reaches a delete action.
6. B - the file must not be downloadable, so writable data lives beside the code and outside the document root.

Cheat sheet

Responses shown in this volume are illustrative for PHP 8.3; version numbers, timestamps and random values differ on every machine.

14.1 Command line

Command	Meaning
`php script.php`	run a script
`php -S localhost:8000 -t public`	start the built-in server
`php -S localhost:8000 router.php`	server with a custom router script
`php -l file.php`	lint, report parse errors only
`php -m`	list loaded extensions

Command	Meaning
`php --ini`	show the configuration files in use
`php -d xdebug.mode=develop script.php`	run with one directive overridden
`php -r 'echo PHP_VERSION;'`	run a snippet without a file

14.2 Language essentials

Topic	Shape
Strict types	`declare(strict_types=1);` as the first statement
Typed function	`function slug(string $t): string`
Nullable	`?string`, unions such as `string` or `int`, any is `mixed`
Promotion	`__construct(private readonly string $id)`
Readonly	assigned once, from the declaring scope
Backed enum	`enum Category: string { case Docs = 'docs'; }`
Pure enum	`enum Suit { case Hearts; }`
`match`	`match ($s) { A => 1, default => 0 }`
Named arguments	`new Link(title: 'x', url: 'y', category: Category::Docs)`
Nullsafe	`$order?->customer()?->name()`
First-class callable	`$controller->index(...)`
Generator	`function rows(): \Generator { yield $r; }`

14.3 Strings, arrays and dates

Need	Use
Length in characters	`mb_strlen($s)`
Cut by characters	`mb_substr($s, 0, 20)`
Split by characters	`mb_str_split($s, 5)`
Escape for HTML	`htmlspecialchars($s, ENT_QUOTES, 'UTF-8')`
Slug	`preg_replace('/[^a-z0-9]+/', '-', strtolower($s))`
JSON	`json_encode($v, JSON_PRETTY_PRINT)` plus the unescaped-slashes flag
Immutable date	`new DateTimeImmutable('2026-10-10 09:00')`
Add time	`$date->modify('+3 days')` returns a new object
Difference	`$a->diff($b)->days`, `->h`, `->i`

Need	Use
Format	`$date->format('D, d M Y H:i')`

14.4 Composer

Command	Meaning
`composer install`	install exactly the locked versions
`composer update`	move inside constraints, rewrite the lock
`composer require vendor/pkg`	add a dependency and install it
`composer require --dev vendor/pkg`	add a development dependency
`composer dump-autoload -o`	rebuild an optimised autoloader
`composer run-script test`	run a script from `scripts`
`composer validate`	check the manifest
`composer audit`	advisories for the locked versions

Constraint	Range
`^3.6`	`>=3.6.0 <4.0.0`
`~3.6.0`	`>=3.6.0 <3.7.0`
`3.*`	any `3.x`

14.5 PDO

Task	Code
Connect	`new PDO($dsn, $user, $pass)`
Exceptions	`setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION)`
Prepare	`$pdo->prepare('... WHERE id = :id')`
Execute	`$st->execute(['id' => $id])`
Many rows	`->fetchAll(PDO::FETCH_ASSOC)`
One value	`->fetchColumn()`
Row count	`$st->rowCount()`
Last id	`$pdo->lastInsertId()`
Transaction	`beginTransaction()`, `commit()`, `rollBack()`

14.6 Errors, testing, security, deployment

Area	Remember
Catch order	most specific class first
Unhandled exception	`set_exception_handler()`
Warning to exception	`set_error_handler()` plus `ErrorException`
Production reporting	`display_errors=Off`, `log_errors=On`
Strict assertion	`assertSame()`
Exception test	`expectException()` before the throwing line
Data provider	a static method returning an array of rows
Password	`password_hash()`, `password_verify()`, `password_needs_rehash()`
Token compare	`hash_equals()`
Server install	`composer install --no-dev --optimize-autoloader`
Opcode cache	`opcache.enable=1`, reset on deploy
Secret storage	environment variable or a file outside the web root

Final test

1. Which command-line flag parses a PHP file without running it? A) `php -m file.php` B) `php -l file.php` C) `php -i file.php` D) `php -S file.php`
2. What does `declare(strict_types=1);` change in a file?
3. With the prefix `App\` mapped to `src/`, which file holds `App\Store\Cart`? A) `src/App/Store/Cart.php` B) `src/Store/Cart.php` C) `Store/Cart.php` D) `src/Store/Cart.class`
4. Where does PSR-12 put the opening brace of a class? A) on the same line as the class name B) on the line after the class name C) indented below the class name D) anywhere, as long as a tool can fix it
5. How many times may a readonly property be assigned? A) as often as the code wants B) exactly once, from the declaring scope C) once per request D) only if the class is not `final`
6. What is the difference between a pure and a backed enum?
7. Which comparison does `match` use, and what happens when no arm matches?
8. Inside a namespace, how do you refer to the global class `DateTimeImmutable`? A) `DateTimeImmutable` B) `\DateTimeImmutable` C) `global\DateTimeImmutable` D) `::DateTimeImmutable`
9. Which SPL structure models an undo history? A) `SplQueue` B) `SplStack` C) `ArrayObject` D) `SplFixedArray`
10. Which keyword inside a function body makes the function a generator? A) `yield` B) `await` C) `defer` D) `emit`
11. What is the difference between `composer install` and `composer update`?

12. Which versions does the constraint `^2.4` accept? A) `2.4` only B) `>=2.4` and `<3.0` C) `>=2.4` and `<2.5` D) any `2.x`
13. Which section of `composer.json` holds the PSR-4 prefixes? A) `require` B) `autoload` C) `scripts` D) `config`
14. What does a repository hide from the rest of the application? A) the HTTP layer B) the storage mechanism C) the templates D) the router
15. True or false: constructor injection is the preferred way to give a class its collaborators.
16. Which PDO attribute turns query errors into exceptions? A) `PDO::ATTR_DEFAULT_FETCH_MODE` B) `PDO::ATTR_ERRMODE` with `PDO::ERRMODE_EXCEPTION` C) `PDO::ATTR_EMULATE_PREPARES` D) `PDO::ATTR_STRINGIFY_FETCHES`
17. Which fetch mode returns each row as an associative array? A) `PDO::FETCH_NUM` B) `PDO::FETCH_OBJ` C) `PDO::FETCH_ASSOC` D) `PDO::FETCH_CLASS`
18. Why can a crafted string not turn a prepared statement into another query?
19. What must a `catch` block do when an exception is thrown inside a transaction? A) nothing, the transaction closes by itself B) call `rollBack()` C) call `commit()` D) close the connection
20. Which function converts an ampersand into the `&` entity for HTML? A) `htmlentities()` B) `htmlspecialchars()` C) `strip_tags()` D) `nl2br()`
21. Which function creates a password hash? A) `password_hash()` B) `hash_password()` C) `password_encode()` D) `crypt_password()`
22. Why is `hash_equals()` used when comparing a CSRF token?
23. What must happen to the session immediately after a successful login, and why?
24. Which PHPUnit assertion checks both type and value? A) `assertEquals()` B) `assertSame()` C) `assertEquivalent()` D) `assertSimilar()`
25. Write the two lines that insert a title into `links` with a named placeholder and execute it.

15.1 Final test answers

1. B - `php -l` parses the file and reports syntax errors only, which is the cheapest check before a commit.
2. It makes scalar type declarations strict for that file: a call that passes a value of the wrong type throws a `TypeError` instead of being coerced.
3. B - the prefix is stripped and the remaining backslashes become directory separators, giving `src/Store/Cart.php`.
4. B - classes and methods put the brace on the following line; control structures keep it on the same line.
5. B - the single write happens during initialisation, and a later assignment raises an `Error`.
6. A pure enum has only cases (`Suit::Hearts`); a backed enum declares a type and a value per case, and offers `->value` and `from()`.
7. `===`, so no type juggling; when nothing matches PHP throws `UnhandledMatchError` instead of falling through.

8. B - class names are not resolved against the global namespace inside a namespace, so the name needs the leading backslash or an import.
9. B - a stack pops the newest item first, which is the order undo needs.
10. A - `yield` makes the function return a generator that produces values one at a time.
11. `install` installs what the lock file pins and is reproducible; `update` recalculates inside the constraints and rewrites the lock file.
12. B - the caret accepts the same major version, at or above the stated minor.
13. B - `autoload.psr-4` maps prefixes to directories.
14. B - callers ask for operations; they never learn which storage answers.
15. True - a constructor argument is required and visible, so a dependency cannot be hidden and the class stays testable.
16. B - `ATTR_ERRMODE` set to `ERRMODE_EXCEPTION` turns every database error into a thrown `PDOException`.
17. C - the associative fetch mode maps column names to values.
18. The database parses the statement first and receives values afterwards as bound parameters, so an input can never change the statement's structure.
19. B - `rollBack()` discards the statements of the open transaction and leaves the connection usable.
20. B - `htmlspecialchars()` converts the characters that carry special meaning in HTML, with the charset given explicitly.
21. A - `password_hash($password, PASSWORD_DEFAULT)` picks the current algorithm and generates a random salt.
22. Because it compares in constant time, so a mismatch position cannot be discovered by timing many attempts.
23. `session_regenerate_id(true)`: a fresh identifier is issued and the old one deleted, which prevents session fixation after the login.
24. B - `assertSame()` fails when the types differ, which catches mistakes that `assertEquals()` accepts.
25. `$statement = $pdo->prepare('INSERT INTO links (title) VALUES (:title)');` followed by `$statement->execute(['title' => $title]);`

Sources

Consulted while writing this volume:

- PHP The Right Way, the syllabus source: https://phptherightway.com/
- Welcome and how the guide is maintained: https://phptherightway.com/#welcome
- Getting started, versions and the built-in server: https://phptherightway.com/#getting_started
- Code style guide, PSR-1, PSR-12 and PSR-4: https://phptherightway.com/#code_style_guide

- Language highlights, paradigms, namespaces, SPL and the CLI: https://phptherightway.com/#language_highlights
- Dependency management with Composer: https://phptherightway.com/#dependency_management
- Coding practices, dates, UTF-8 and design patterns: https://phptherightway.com/#coding_practices
- Dependency injection: https://phptherightway.com/#dependency_injection
- Databases through PDO: https://phptherightway.com/#databases
- Templating with plain PHP and compiled templates: https://phptherightway.com/#templating
- Errors and exceptions: https://phptherightway.com/#errors_and_exceptions
- Web application security: https://phptherightway.com/#security
- Testing: https://phptherightway.com/#testing
- Servers and deployment: https://phptherightway.com/#servers_and_deployment
- Caching: https://phptherightway.com/#caching
- Documenting your code: https://phptherightway.com/#documenting
- PHP manual, language reference index: https://www.php.net/manual/en/langref.php
- PHP manual, classes and objects: https://www.php.net/manual/en/language.oop5.php
- PHP manual, enumerations: https://www.php.net/manual/en/language.enumerations.php
- PHP manual, type declarations: https://www.php.net/manual/en/language.types.declarations.php
- PHP manual, namespaces: https://www.php.net/manual/en/language.namespaces.php
- PHP manual, generators: https://www.php.net/manual/en/class.generator.php
- PHP manual, exceptions: https://www.php.net/manual/en/language.exceptions.php
- PHP manual, PDO: https://www.php.net/manual/en/book.pdo.php
- PHP manual, prepared statements and stored procedures: https://www.php.net/manual/en/pdo.prepared-statements.php
- PHP manual, transactions: https://www.php.net/manual/en/pdo.transactions.php
- PHP manual, password hashing: https://www.php.net/manual/en/function.password-hash.php
- PHP manual, data filtering: https://www.php.net/manual/en/function.filter-var.php
- PHP manual, output escaping: https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP manual, DateTimeImmutable: https://www.php.net/manual/en/class.datetimeimmutable.php
- PHP manual, multibyte strings: https://www.php.net/manual/en/book.mbstring.php
- PHP manual, command line usage: https://www.php.net/manual/en/features.commandline.php
- PHP manual, the built-in web server: https://www.php.net/manual/en/features.commandline.webserver.php

- PHP manual, core configuration directives: https://www.php.net/manual/en/ini.core.php
- Composer, basic usage: https://getcomposer.org/doc/01-basic-usage.md
- Composer, the manifest schema: https://getcomposer.org/doc/04-schema.md
- Composer, version constraints: https://getcomposer.org/doc/articles/versions.md
- Composer, command line interface: https://getcomposer.org/doc/03-cli.md
- PSR-1, basic coding standard: https://www.php-fig.org/psr/psr-1/
- PSR-12, extended coding style: https://www.php-fig.org/psr/psr-12/
- PSR-4, autoloading: https://www.php-fig.org/psr/psr-4/
- PSR-3, logger interface: https://www.php-fig.org/psr/psr-3/
- PSR-11, container interface: https://www.php-fig.org/psr/psr-11/
- PHPUnit documentation: https://docs.phpunit.de/en/10.5/
- Twig documentation: https://twig.symfony.com/doc/3.x/
- Packagist, the package registry: https://packagist.org/